TOWSON UNIVERSITY
COLLEGE OF GRADUATE STUDIES

FROM ASSIGNMENT DOCUMENTS TO REPRODUCIBLE DEVELOPMENT ENVIRONMENTS: AN AI-ASSISTED APPROACH FOR COMPUTER SCIENCE EDUCATION.

by

Luis Dale Gascon

A thesis
Presented to the faculty
of Towson University
in partial fulfillment
of the requirements for the degree
Master of Science
Department of Computer and Information Sciences

Towson University,
Towson, Maryland
21252

December 2026

Docusign Envelope ID: 24F1B278-468C-4F3E-91E7-E942561CC002

TOWSON UNIVERSITY | Office of Graduate Studies

Thesis Committee Approval Form

Student's Name

Luis Dale Gascon

Chairperson, Thesis Committee DocuSigned by: Sajidul Islam A1834C6028884BD... (Signature) Md Sajidul Islam Sajid (Typed name)

Note: Please attach a description of the affiliation and credentials of any non- Towson University members of the Committee, and the members' *curriculum vitae.*

Member Signed by: Sidd Kaza 1C79FD345A714C7... (Signature) Sidd Kaza (Typed name)

Member DocuSigned by: Blair Taylor F736E4AFF5A443D... (Signature) Blair Taylor (Typed name)

Approved by

Graduate Program Director

Signed by: Eiying Tang C6D51D455F89448... (Signature) 02/02/26 (Date)

Department Chairperson

DocuSigned by: Michael McGuire 4FA74552EDAD4F1... (Signature) 02/02/26 (Date)

☒ Graduate Faculty Status verified for all committee members

Dean of Graduate Studies

Signed by: David R Ownby 0109A385FE4043B... (Signature) 02/03/26 (Date)

Acknowledgments

I would like to express my sincere gratitude to Dr. Taylor and Dr. Kaza for their continuous support throughout my graduate studies. I will always be thankful for the opportunities that they have provided.

Special thanks go to my thesis advisor, Dr. Sajid, for generously agreeing to advise this work on short notice and for dedicating his time to reviewing my drafts under a tight deadline. His insightful suggestions greatly strengthened this thesis, and I deeply appreciate his willingness to invest the time and effort to engage with such an esoteric subject matter.

I would also like to thank the student volunteers, whom I endearingly referred to as "my lab rats," for participating in the onboarding experiments. Their time, patience, and candid feedback were invaluable to evaluating the practical ergonomics of this work.

Beyond the lab, I am grateful to the Outdoor Adventure Club and its climbing wall community for providing a much-needed outlet away from my research. The friendships I formed there kept me grounded and sane throughout my graduate program.

Finally, I am deeply grateful to my parents for their unconditional belief in my potential and their sacrifices on my behalf. I would not be where I am today without their steadfast love and encouragement.

Abstract

FROM ASSIGNMENT DOCUMENTS TO REPRODUCIBLE DEVELOPMENT ENVIRONMENTS: AN AI-ASSISTED APPROACH FOR COMPUTER SCIENCE EDUCATION.

Luis Dale Gascon

In computer science courses, students frequently spend substantial time configuring software toolchains and debugging installation errors rather than working on course assignments, while instructors face significant difficulties supporting a heterogeneous array of student machines across macOS, Linux, and Windows. This thesis presents an approach that enables instructors to transform existing coursework materials into reproducible, declarative development environments leveraging Nix Flakes. Powered by a custom agent skill (`course-flake-generator`) running within an instructor's local AI agent harness, the workflow progresses through six stages: ingesting an instructor's assignment document or starter codebase, extracting requirements using skill-bundled scripts, generating a `flake.nix` from modular templates, validating the configuration through closed-loop compiler checks, producing an instructor-approved environment with a cryptographically pinned `flake.lock`, and enabling single-command student activation via `nix develop`. Empirical evaluations across software development and data science benchmarks demonstrate that structured skill scaffolding and compiler feedback enable agents to reliably synthesize functional development shells (reaching up to 97.62% Pass@1 and 100.00%

Pass@3). Furthermore, ablation evaluations reveal that while `mcp-nixos` serves as the primary registry discovery mechanism, the skill's native Nix CLI fallback performs comparably for standard coursework dependencies. Automated test scripts across unmounted Linux containers and macOS virtual machines verify cross-platform determinism and expose hidden authoring-host assumptions before classroom release, while a controlled student onboarding study demonstrates that single-command environment activation eliminates procedural errors common to manual installation guides. Finally, this thesis examines the operational, adoption, and supply-chain security considerations of deploying declarative environments in computer science curricula.

Table of Contents

List of Tables ix

List of Figures x

List of Listings xi

Chapter I. Introduction 1

A. The Environment Configuration Problem in Computer Science Education 1

B. Virtualization versus Declarative Configuration 2

C. Proposed Approach: An Agent Skill for Assignment-to-Flake Synthesis 5

D. Research Objectives and Contributions 8

E. Organization of the Thesis 10

Chapter II. Background and Literature Review 11

A. Related Work 11

1) Standardizing Educational Environments via Virtual Machines and Containers 11

2) Learning Opportunity Provided from Managing Systems 13

3) Declarative and Functional Package Management in Education 13

4) Synthesis of Infrastructure as Code Using Large Language Models 15

5) Agentic Skills and Tool-Augmented Assistance 16

B. Nix Foundations 17
1) Declarative Packages and Nixpkgs 18
2) The Nix Store and Dependency Closures 19
3) Flakes, Locked Inputs, and Development Shells 20
4) Realization, Binary Caches, and Trust 23
C. Summary of Research Gaps 24
Chapter III. Research Questions and Proposed Architecture 27
A. Research Questions and Hypotheses 27
B. Proposed Solution: An agent skill for AI-Assisted Nix Flake Synthesis 28
Chapter IV. Experimental Methodology and Test Setup 33
A. Agentic Flake Synthesis Evaluation Protocol 33
1) Benchmark Dataset Curation and Evaluation Suites 33
2) Agent Harness, Model Configurations, and Tooling Conditions 36
3) Three-Stage Validation Pipeline and Pass@K Formulation 42
B. Clean-Environment Reproducibility Protocol 43
1) Guest Replay Pipeline and Failure Taxonomy 44
C. Student Onboarding Ergonomics Protocol 45
D. Experimental Testbed and System Infrastructure 47
Chapter V. Empirical Results and Analysis 49
A. Flake Synthesis Accuracy and Model Comparison 49
1) Codebase-Inferred and Document-Based Flake Synthesis 52
B. Tool Invocation and Telemetry Analysis 56

1) Native Nix CLI Discovery versus MCP Retrieval 56
C. Base OS Reproducibility Results 63
D. Student Onboarding Ergonomics Results 69
1) Installation of the Rust Compiler 69
a) Student A 69
b) Student B 70
2) Jupyter Notebook 71
3) Full Stack Environment 71
Chapter VI. Conclusion 75
A. Summary of Contributions 75
B. Synthesis of Research Questions and Hypotheses 78
C. Discussion: Operational Constraints and Deployment Limitations ... 80
1) Adoption of Nix Flakes 80
2) Operational Availability and Maintenance Conditions 84
3) Security Considerations and Binary Cache Trust Boundaries .. 85
D. Limitations and Future Work 87
Appendix A Disclosure of Generative AI Use 89
A. AI-Assistance 89
B. AI-Generated Artifacts 90
1) AI-Generated Input for RQ1 94
References 96

List of Tables

Table I Comparison of Development Environment Management Approaches 26

Table II Host Machine Hardware and System Environment 47

Table III Flake Synthesis Accuracy Across Models and MCP Conditions 51

Table IV Mean Trial Duration and Token Consumption Across Models and MCP Conditions 51

Table V Package Precision and Over-Provisioning Ratio Across Passing Trials 51

Table VI Synthesis Performance on Existing Coursework Codebases 52

Table VII Synthesis Performance on Instructional Course Documents 52

Table VIII MCP Tool Invocation Comparison in MCP-Enabled Evaluations 62

Table IX Onboarding Results for Rust Toolchain Setup 69

Table X Onboarding Results for Jupyter Notebook Setup..... 71

Table XI Onboarding Results for Full Stack Environment Setup 72

List of Figures

Fig. 1 End-to-End Architecture of the Autonomous `flake.nix` Synthesis Pipeline Icons created by Magnific - Flaticon 32

Fig. 2 MCP Tool Invocations Across Benchmark Trials: Gemini 3.8 Flash versus GPT-OSS 120B 61

Fig. 3 Linux Container Replay Reproducibility Outcomes 65

Fig. 4 macOS Virtual Machine Replay Reproducibility Outcomes 65

Fig. 5 Student Onboarding Completion Times Across Coursework Environments 72

Fig. 6 Average Onboarding Time Delta by Coursework Environment Task ... 73

List of Listings

Listing 1 Example of a flake file 20

Listing 2 `flake.lock` `nixpkgs` Attributes 22

Listing 3 Evaluation Metadata Format and Ground-Truth Schema 35

Listing 4 Custom Skill Filetree Structure 36

Listing 5 assets/flake-devshell.template.md template 37

Listing 6 Benchmark Execution Output Directory Structure 39

Listing 7 Example `nix search` Output for Package Discovery 57

Listing 8 Natural Language Instruction Benchmark Inputs 94

Chapter I.

Introduction

A. The Environment Configuration Problem in Computer Science Education

In computer science education, programming assignments are designed to teach algorithmic thinking, systems design, and domain-specific problem solving; yet in practice, students frequently spend a disproportionate share of their assignment time configuring development environments rather than writing code [1–3]. Modern university classrooms encompass a highly heterogeneous array of personal student laptops spanning macOS, disparate Linux distributions, and Microsoft Windows [1, 3]. Because introductory and advanced courses alike depend on specialized compilers, language runtimes, software development kits (SDKs), and native libraries, minor variations across student operating systems routinely trigger installation failures, path misconfigurations, and library version conflicts before a student can begin the actual coursework.

For instructors and teaching assistants, this operating system diversity creates an acute support burden. Historically, course staff have relied on imperative, step-by-step installation guides distributed at the start of a semester or alongside individual assignments [1, 2]. When these manual instructions encounter preexisting packages, non-standard shell profiles, or operating system updates on student machines, lecture periods, laboratory sessions, and office hours are diverted from computer science pedagogy into ad-hoc technical support [1, 3]. Commercial

software engineering organizations mitigate this friction through dedicated platform engineering teams, standardized corporate hardware, and managed developer images [4–6]. In contrast, academic instructors must support students working on unmanaged personal devices with widely varying system configurations.

Holland et al. [7] characterize this manual, trial-and-error tinkering with system state as "futzing," showing that mutable operating system and application state is a primary source of software fragility in which a single misordered command or conflicting package installation can leave a machine in a broken intermediate state. Because recovering from such failures requires system administration expertise that students are still developing [2, 7], environment configuration friction acts as an unproductive barrier that consumes student time, delays assignment progress, and undermines equitable participation across the curriculum.

B. Virtualization versus Declarative Configuration

The root cause of configuration fragility lies in the imperative architecture of conventional graphical installers and package managers. Tools such as Debian's `apt`, Arch's `pacman`, and macOS `brew` destructively mutate a shared global filesystem hierarchy, overwriting files in directories such as `/usr/bin` and `/usr/lib` in place [4, 8]. Because packages share mutable namespaces, upgrading a library required for one course frequently breaks the dependencies of another course, producing a web of dependency conflicts [4, 7]. Furthermore, because imperative package managers do not support atomic upgrades or preserve transactional

execution histories, rolling back failed upgrades to a known working state is rarely feasible without full reinstallation [4, 7].

To overcome the fragility of mutable host filesystems, the software industry and computer science educators widely adopted containerization via Docker and Open Container Initiative (OCI) Development Containers (DevContainers) [1, 3]. DevContainers isolate the application runtime, compiler toolchains, and editor extensions within a sandboxed Linux container, aiming to give students a consistent environment across host operating systems [1]. However, container images built via imperative `Dockerfile` instructions still suffer from unpinned upstream package layers and impose heavy background virtualization overhead on macOS and Windows hosts [1, 8, 9]. Moreover, a locally successful configuration does not establish that an environment will work on another machine: its apparent success may depend on packages, configuration, credentials, or cached build artifacts already present on the authoring host.

In contrast to conventional container workflows, purely functional package management offers an explicit, host-native model for specifying dependencies [5, 10]. The Nix package manager treats software builds as pure functions of their declared inputs, storing every built package under a unique cryptographic hash in an immutable directory known as the Nix store (`/nix/store`) [4, 11]. Nix Flakes standardize this model by providing a hermetic interface governed by a `flake.nix` specification and a pinned `flake.lock` dependency file, which empirical studies have shown to support high build reproducibility across time and platforms [8, 12, 13].

Through ephemeral development shells (`devShells`), students can activate a fully configured environment with a single shell invocation (`nix develop`) and deactivate the same development shell without incurring side effects on their host system [8, 14]. This isolation is especially beneficial for students enrolled concurrently in multiple computer science courses with differing or conflicting technical requirements. Rather than downloading and managing disparate applications, database servers, and runtime version managers that risk clashing on the global filesystem, students need only install Nix once. The flake file associated with each course repository cleanly isolates and manages all required dependencies, allowing students to switch between course environments without cross-course interference. The explicit specification and lock file also make Nix Flakes a suitable basis for testing whether an agent-generated environment can be reproduced from a clean host, rather than merely validated where it was authored.

Chapter II provides the technical background on Nix and how it stacks against its alternatives. It reviews existing approaches to educational environment management, identifies their operational bottlenecks, and establishes the foundational concepts of functional package management.

Despite these compelling architectural advantages, Nix Flakes have faced an acute adoption chasm in educational institutions [8, 14]. The Nix expression language is a domain-specific, lazy functional programming language that departs radically from conventional imperative scripting [8, 15]. Expecting instructors to manually author and maintain multi-architecture `flake.nix` specifications for every

course assignment, or expecting students to diagnose cryptic Nix evaluation traces, merely trades one form of configuration burden for another [6, 8, 16]. Consequently, instructors remain tethered to fragile manual installation guides or container workflows that demand background Linux virtual machines and substantial memory overhead on non-Linux laptops.

C. Proposed Approach: An Agent Skill for Assignment-to-Flake Synthesis

What if an instructor could simply provide an existing assignment document to an AI agent that automatically extracts requirements, generates a reproducible development environment, and verifies its validity? The instructor could then review and distribute that verified configuration to the class, allowing students to activate a fully provisioned development environment through a single text file (`flake.nix`) and a single command-line invocation (`nix develop`) rather than manual imperative setup.

This thesis realizes that vision through `course-flake-generator`, a custom agent skill that enables instructors to transform existing course materials into ready-to-use Nix Flake development environments using their own local AI agent harness [8, 17, 18]. Rather than requiring instructors or students to master the Nix expression language, the skill equips the instructor's local agent harness to execute a structured assignment-to-flake workflow spanning requirement extraction, flake generation, closed-loop compiler validation, instructor approval, and student activation.

First, the workflow begins with an instructor's existing assignment materials, such as a PDF, Microsoft Word handout, a README.md file, or unconfigured starter codebase. Second, the custom agent skill provides extraction scripts and heuristics that inspect the assignment materials and scan project manifests to infer required compilers, language runtimes, build tools, and libraries. Third, the skill guides the agent using curated multi-architecture templates to synthesize a declarative `flake.nix` specification tailored to the assignment. Fourth, the candidate flake undergoes automated validation, where skill-bundled validation routines execute closed-loop evaluation and shell realization checks to catch syntax errors or missing dependencies and produce a cryptographically pinned `flake.lock` file. Fifth, the instructor reviews the verified configuration and commits it to the assignment repository. Finally, students clone or download the repository onto their personal machines and invoke a single command (`nix develop`) to enter a reproducible development shell and immediately begin working on the assignment.

This workflow is especially advantageous for instructors managing semester-long courses. Instead of maintaining per-platform setup scripts or fielding repetitive installation guides, instructors can distribute a single, self-contained configuration file alongside their assignment repository. Teaching assistants and lab instructors are thereby freed from manual troubleshooting, allowing course staff to dedicate their office hours to algorithmic guidance and conceptual assistance.

Deploying off-the-shelf Large Language Models (LLMs) without specialized scaffolding exposes critical limitations for environment generation. Recent

benchmarks on automated Infrastructure as Code (IaC) synthesis show that general-purpose LLMs achieve low first-attempt pass and deployability rates when generating declarative configurations without iterative verification [19, 20]. Furthermore, because the Nix expression language represents a small fraction of public training corpora compared to ubiquitous languages such as Python or JavaScript, unassisted models suffer from elevated package hallucination rates [21, 22]. Unassisted models frequently fabricate non-existent Nixpkgs attribute names, reference deprecated configuration schemas, or default to imperative host installation commands (such as `brew install`) that defeat cross-platform reproducibility.

To address these vulnerabilities, our `course-flake-generator` skill packages domain-specific procedural knowledge into an open, modular format consisting of structured instructions (`SKILL.md`), multi-architecture templates, and helper scripts [23, 24]. By loading this procedural context dynamically within the instructor's local agent harness [17, 24], the agent reliably extracts requirements and generates valid declarative environments.

To resolve and verify package attributes deterministically during flake generation, the skill's primary package discovery mechanism queries a dedicated Model Context Protocol (MCP) [25] server, `mcp-nixos`, which connects directly to the real-time Elasticsearch index powering `search.nixos.org`. When the MCP server is unavailable or disabled, the skill provides a fallback procedure using native Nix CLI inspection tools (such as `nix search` and `nix eval`). Crucially, rather than relying on package search alone, the workflow exploits the deterministic nature of

Nix evaluation: when a generated flake contains syntax or typing errors, compiler diagnostics from `nix flake check` and `nix develop` are captured and fed back into the agent harness in a closed-loop recovery cycle [17, 20]. This feedback loop allows the agent to iteratively self-correct invalid declarations prior to finalizing the generated flake.

D. Research Objectives and Contributions

The primary objective of this thesis is to eliminate the unproductive time students spend configuring local development environments and to alleviate the burden instructors face in supporting diverse student machines. By equipping an instructor's AI agent harness with a custom agent skill to synthesize validated, instructor-approved Nix Flake environments directly from assignment materials, we investigate whether computer science courses can achieve deterministic, cross-platform environment reproducibility while providing students with a frictionless, single-command onboarding experience.

To evaluate each phase of the proposed assignment-to-flake workflow (from agentic requirement extraction and generation in Stages 1–4, to cross-platform portability of the instructor-approved environment in Stage 5, to student activation in practice in Stage 6), this research pursues three primary research questions. These questions systematically investigate synthesis reliability, cross-platform reproduction fidelity, and student onboarding ergonomics:

1. **RQ1:** Can AI automatically generate functional development environments from CS assignment requirements?

2. **RQ2:** Can those generated environments be reproduced consistently across different computing platforms?

3. **RQ3:** What are the potential benefits and practical limitations of this approach for student onboarding?

In answering these questions, this thesis makes four concrete contributions to computer science education and automated environment synthesis. Together, these contributions bridge the gap between declarative package management and practical classroom deployment:

1. We design and implement `course-flake-generator` (`https://github.com/curlyLasagna/Thesis-Flake-Generator`), an open-standard agent skill that equips an instructor's AI agent harness to extract software requirements from assignment documents and existing codebases, generate `flake.nix` files, validate them in a closed loop, and produce pinned `flake.lock` files ready for classroom distribution.

2. Across a two-track computer science curriculum benchmark (Software Development and Data Science), we evaluate how procedural skill scaffolding, closed-loop compiler verification, and package discovery strategies (evaluating the primary `mcp-nixos` registry server alongside a native Nix CLI fallback condition) influence synthesis accuracy (Pass@k), package precision, and dependency over-provisioning across frontier and open-weight models.

3. We develop a script to automate tests of the generated flakes across fresh, unmounted Linux containers and macOS virtual machines, quantifying cross-platform reproduction rates and exposing hidden authoring-host assumptions before environments reach student laptops.

4. Through a controlled onboarding study on clean virtual machines, we compare single-command student activation (`nix develop`) against manual installation from assignment guides in terms of setup time deltas, procedural error prevention, and cognitive friction, complemented by an operational and security analysis of classroom deployment.

E. Organization of the Thesis

The remainder of this thesis is organized as follows. Chapter II reviews related literature in software environment standardization, infrastructure-as-code generation, and functional package management, and establishes the Nix foundations required for the study. Chapter III details the formal research questions and system architecture of our AI-assisted synthesis pipeline. Chapter IV describes the experimental methodology, dataset construction, clean-container replay protocol, and student onboarding study. Chapter V analyzes the empirical results, comparing model synthesis accuracy, tool utilization patterns, clean-container reproduction outcomes, and student onboarding ergonomics. Finally, Chapter VI summarizes the empirical findings of this thesis, examines operational limitations of Nix, challenges surrounding classroom adoption, and software supply-chain security considerations in Section C, and highlights promising avenues for future research.

Chapter II.

Background and Literature Review

A. Related Work

1) Standardizing Educational Environments via Virtual Machines and Containers: Standardizing student development environments has remained a persistent pedagogical challenge in computer science education [2, 26]. In traditional coursework, students frequently encounter installation failures, library mismatches, and operating system incompatibilities when attempting to manually configure personal machines for laboratory assignments. Installation failures may hinder a student's drive to continue learning, consuming significant classroom troubleshooting time and potentially leading less experienced students to conclude that programming is not for them before writing a single line of code [2, 3].

Historically, institutions attempted to resolve configuration discrepancies by deploying centralized compute clusters or multi-user Linux servers accessible via SSH or remote desktop protocols [3]. Both Valstar et al. [1] and Malan [3] critically analyze the friction inherent in these pre-configured central servers, noting that because instructors lack administrative privileges to install applications, they must continually coordinate with central IT personnel who manage infrastructure across disparate academic departments. To bypass these restrictions, Malan [3] notes that instructors often had to resort to compiling programs from source into shared course directories. This administrative bottleneck results in sluggish response times when

course toolchains require updates during active semesters, with Valstar et al. [1] observing multi-week reconfiguration delays that left laboratory software versions up to five years out of date.

Furthermore, centralized remote architectures impose rigid network requirements [1]. Students must maintain persistent, low-latency network connections, either on campus or connecting to an institution-provided VPN. Unstable internet connections introduce substantial input latency and unexpected disconnects, severely disrupting the student learning experience during programming assignments and interactive debugging sessions [1]. To alleviate these network and administrative dependencies, educators started incorporating virtual machines into their courses [2, 26]. Virtual machines provide strong isolation and ensure uniform guest environments regardless of the underlying host platform. Virtual machines also allowed instructors to quickly reset or change a machine's state via snapshots without IT personnel present [2, 26]. However, distributing multi-gigabyte virtual disk images imposes steep download times, slow boot sequences, and heavy compute demands on student laptops [3, 26].

To reduce this virtualization overhead, educators shifted toward client-side and cloud-backed docker containerization to provide students with a complete development environment [1, 3]. DevContainers leverage Docker to package compilers, runtimes, and text editor configurations into a version-controlled container specification, ensuring that every enrolled student executes code in an identical software sandbox [1]. Containers were also faster for staff to develop and for students

to launch, starting in seconds rather than the minutes required to boot a full guest operating system [1, 3]. However, this approach still incurs computational and storage overhead on client machines, requiring a background Linux virtual machine on Windows and macOS hosts and consuming significant disk space and working memory [1].

2) Learning Opportunity Provided from Managing Systems: Despite the advantages that pre-configured containers provide, completely abstracting system configuration can also remove valuable hands-on exposure when system management itself is a pedagogical goal [2]. Harvie et al. [2] note that deliberately requiring students to install server packages via Ubuntu's Advanced Packaging Tool (APT) and configure network ports inside a baseline virtual machine provided a practical learning opportunity that improved initial student comprehension of package management and client-server architectures. Conversely, in domain-focused courses and cybersecurity exercises where environment setup is merely a prerequisite rather than the learning objective, Gupta et al. [27] demonstrate that requiring learners to manually reconstruct and rehost complex software environments creates a substantial entry barrier that diverts focus from conceptual mastery to technical troubleshooting.

3) Declarative and Functional Package Management in Education: To circumvent the heavy computational overhead of virtualized container engines while preserving reproducibility, educators and researchers have begun exploring purely functional package management via Nix Flakes [8, 14]. Hasanović [14] reports on the migration of university C++ laboratory environments from Docker containers to declarative

Nix Flake environments hosted on a public GitLab repository, allowing students to launch preconfigured Neovim, Visual Studio Code, and interactive C++ REPL development shells with a single command. Similarly, Du et al. [8] demonstrate that Nix Flakes unify C/C++ and Python software stacks under a single declarative specification, providing full environment isolation and cross-project composition in user space without requiring root privileges.

The theoretical strengths of functional software deployment have been documented extensively in research literature [4, 5, 10, 11]. By treating package builds as pure functions whose outputs depend exclusively on cryptographically hashed inputs, Nix aims to guarantee that identical declarations yield identical store paths. Bandt [12] conducts an empirical assessment of reproducibility in Nixpkgs, analyzing how functional package management mitigates non-deterministic build behaviors and how remaining sources of non-determinism can be detected. In scientific and academic computing, Hausch et al. [16] demonstrate through a case study on the preCICE multiphysics ecosystem that Nix/NixOS enables self-contained, reproducible research environments where traditional imperative Vagrant VM provisioning failed due to unpinned upstream dependencies. At the repository scale, Malka et al. [13, 28] empirically validate build-environment reproducibility across space and time, successfully rebuilding 99.94% of 14,000 packages from a six-year-old Nixpkgs revision and demonstrating bitwise reproducibility rates between 69% and 91% across 709,816 historical package builds.

Despite these reproducibility guarantees, adopting Nix introduces a significant usability trade-off. Du et al. [8] emphasize that Nix imposes a relatively steep learning curve because its lazy functional language and packaging conventions are unfamiliar to most students and instructors alike. Hausch et al. [16] similarly observe that packaging complex software often requires workarounds for Nix's isolated `/nix/store` hierarchy that are difficult for non-experienced Nix users to implement, while Yin [6] highlights that sparse official documentation and cryptic evaluation error messages further elevate the entry barrier for newcomers. Notably, Du et al. [8] observe in practice that LLM assistance can help lower this barrier by generating Nix expressions and diagnosing evaluation errors.

4) Synthesis of Infrastructure as Code Using Large Language Models: The rapid proliferation of Large Language Models has spurred widespread investigation into automating Infrastructure as Code (IaC) and system configuration management [29]. Rather than manually composing complex scripts, developers can specify deployment goals in natural language and delegate script generation to automated models. Srivatsa et al. [22] conduct a comprehensive survey evaluating LLM capabilities across major declarative and procedural configuration platforms, noting that limited training data and rapidly evolving cloud APIs frequently cause models to produce syntactically plausible configurations that fail or introduce security risks in real-world deployments. This divergence stems from the models' lack of awareness of current best practices, platform updates, and complex multi-resource dependencies [22].

Recent benchmark studies substantiate these findings. Kon et al. [19] introduce IaC-Eval, demonstrating that contemporary models like GPT-4 achieve only a 19.36% pass@1 accuracy rate when generating declarative cloud infrastructure programs from natural language requirements, compared to 86.6% on standard Python benchmarks. Similarly, Zhang et al. [20] emphasize that deployability, rather than surface syntax correctness, is the decisive metric for automated infrastructure generation, finding that SOTA LLMs achieve only 20.8% to 30.2% first-attempt deployment success unless augmented with iterative verification and deployment feedback loops. A foundational vulnerability in automated code and environment synthesis is package hallucination [21]. Krishna et al. [21] analyze package hallucination across Python, JavaScript, and Rust, demonstrating that LLMs frequently fabricate non-existent package identifiers; particularly in ecosystems with smaller registries and less training data representation such as Rust, where mean hallucination rates reach 24.74% and exceed 42% for several models. In software pipelines, these hallucinations break build executions and expose users to package squatting supply-chain attacks, underscoring the critical need for deterministic external package verification [21].

5) Agentic Skills and Tool-Augmented Assistance: Recent research in agent harness architectures demonstrates that the reliability of autonomous LLMs on complex, multi-step tasks is governed heavily by the surrounding harness; comprising the execution loop, tool registry, context management, and verification interfaces; rather than the base model alone [17, 18]. To structure domain-specific procedural

knowledge within such harnesses, Agent Skills have emerged as modular, reusable packages that combine triggering metadata, workflow instructions, templates, and helper scripts [23, 24]. Ling et al. [24] conduct a large-scale analysis of 40,285 public agent skills, showing that skills are heavily concentrated in software engineering and infrastructure workflows while remaining compact enough (with a median length of 1,414 tokens) to fit within standard prompt budgets without hard-coded orchestration rules. At the same time, security audits of public skill marketplaces reveal that unvetted third-party skills frequently introduce command-execution and data-exfiltration risks [24, 30], motivating carefully curated, sandboxed skill designs.

Integrating standardized tool protocols further strengthens agentic workflows. Inspired by the architectural decoupling of the Language Server Protocol (LSP) for development editors [31], the Model Context Protocol (MCP) defines a unified, bidirectional standard for connecting AI agents with external tools and data sources [25]. Although indiscriminate tool invocation can inflate token costs or degrade performance when unnecessary [18, 32], coupling LLM reasoning with targeted, real-time package registry lookups and compiler verification provides a direct mechanism to ground package selection and mitigate the hallucination vulnerabilities identified in prior studies [20, 21].

B. Nix Foundations

Having reviewed existing approaches and the emerging role of declarative tooling, this section establishes the fundamental Nix concepts required to understand its architectural advantages and operational limitations. We examine the package

derivation model, the cryptographic store architecture, and the flake specification standard that underpin our automated synthesis pipeline.

Nix is both a package manager and a purely functional domain-specific language for describing how software dependencies and runtime environments are constructed. Unlike an imperative installation procedure, which modifies a shared system state step by step, a Nix expression describes the inputs from which a package or environment should be produced. Nix then evaluates that description and creates an immutable derivation. This distinction is central to the thesis: an agent generates a declarative specification rather than a sequence of host-specific installation commands.

1) Declarative Packages and Nixpkgs: In Nix, a *derivation* is a build specification for a package. It identifies the builder, its source inputs, build and run-time dependencies, configuration arguments, and expected outputs. Nix evaluates derivations as part of a dependency graph, allowing a package to depend on specific versions of compilers, libraries, and build tools rather than on whichever versions happen to be installed globally. The functional model aims to make a build result a function of its declared inputs, thereby avoiding hidden dependencies on mutable global directories [4, 10].

The collection of such package definitions is a repository called Nixpkgs. It exposes packages as named attributes, such as `python3`, `cmake`, or `rustc`, and parameterizes many definitions by the target platform. Beyond standalone system binaries, Nixpkgs also provides dedicated language-specific packaging frameworks

and attribute sets, allowing language libraries to be composed declaratively alongside system compilers without mutating user-space package directories [33]. A generated development environment therefore does not install packages by conventional system mutation; it selects attributes from a particular Nixpkgs revision and composes them into a shell or package output. This is why an invalid attribute name, a deprecated package, or a package unavailable on the target platform can prevent an otherwise plausible environment from evaluating. RQ1 evaluates whether the agent can make these selections correctly.

2) The Nix Store and Dependency Closures: Nix places realized build outputs in a global Nix store, conventionally `/nix/store`. Store paths are named with a cryptographic hash followed by a human-readable package name, for example `/nix/store/g33w0qxvkblmbqbpwnxdy1c8shji05gf-gcc-21.0.0`. The hash distinguishes the declared input configuration associated with that output. Store paths are treated as immutable once created, so an upgrade does not overwrite a library used by an existing environment; instead, it creates or selects a distinct store path. This permits multiple versions of a tool or library to coexist without sharing a mutable installation prefix [11].

Every realized package refers to other store paths that it requires at run time. The transitive set of those paths is its *closure*. A development environment is therefore more than a short list of executables: it includes the compiler, libraries, interpreters, shell hooks, and other transitive dependencies needed by those executables. Nix makes this closure explicit and can construct a shell in which

selected package binaries are available through `PATH`. The experimental command `nix develop` is the primary interface used in this thesis to activate what is known as the dev shell.

The store model explains both the benefit and the limitation evaluated by RQ2. A host on which an environment has already been used may contain a large, warmed store whose paths mask missing declarations or make activation appear faster than it would be for a student. The clean-container replay therefore starts each validation stage without a mounted host store or project directory. It tests whether the generated declaration and its transitive closure can be obtained and used independently of the authoring machine's prior state.

3) Flakes, Locked Inputs, and Development Shells: A flake is an experimental convention for organizing a Nix project around two files. The `flake.nix` file declares inputs and outputs. Inputs commonly include Nixpkgs, the Nix package manager or other Git repositories; outputs may include packages, applications, NixOS configurations, or development shells.

```
{
  description = "Go Development Environment";

  # External dependency sources called inputs
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.11";
    flake-utils.url = "github:numtide/flake-utils";
  };

  # Build outputs
  outputs =
    {
      nixpkgs,
      flake-utils,
      ...
    }:
```

```
    # Generate outputs across standard target architectures (e.g., x86_64-linux, aarch64-darwin
    )
    flake-utils.lib.eachDefaultSystem (
      system:
      let
        # Instantiate the Nixpkgs package set for the target system
        pkgs = import nixpkgs {
          inherit system;
          config.allowUnfree = true;
        };
      in
      {
        # Ephemeral development environment activated via `nix develop`
        devShells.default = pkgs.mkShell {
          # Packages added to the shell's PATH
          packages = with pkgs; [
            go
          ];

          # Commands executed automatically upon entering the shell
          shellHook = ''
            echo "Entering environment for Go Development Environment"
          '';
        };
      }
    );
}
```

Listing 1. Example of a flake file

In this thesis, the relevant output is a `devShell`, which lists the tools and environment configuration required by a course assignment. A student activates that output with the `nix develop` command, which instantiates an ephemeral subshell containing all declared dependencies and environment variables from the generated `flake.nix` file.

The companion `flake.lock` file records resolved input revisions and associated metadata. It represents a commitment to a specific dependency graph: once an instructor reviews and commits it, students use the same pinned inputs

rather than silently receiving the newest revision of Nixpkgs. Updating an input with `nix flake update` is therefore a maintenance action that intentionally changes the lock file; it is not part of ordinary activation of a fixed course environment. Pinning supports repeatability across student machines, but a lock file is not an offline archive. If an input or store path is absent locally, Nix may still need network access to retrieve it.

```
"nixpkgs": {
  "locked": {
    "lastModified": 1791011291,
    "narHash": "sha256-V6OBJccNKBo/ktknxlJXZChPoI0NE2lembArBO3CE9k=",
    "owner": "NixOS",
    "repo": "nixpkgs",
    "rev": "73e728ddb6b7a12d18808f510813a13ee1fe4cce",
    "type": "github"
  },
}
```

Listing 2. `flake.lock` `nixpkgs` Attributes

Here, the `rev` attribute designates the exact Git commit hash of the Nixpkgs repository pinned for the project, while `narHash` records the cryptographic hash of the serialized Nix Archive (NAR) content [34]. By locking this commit and content hash, Nix ensures that all downstream student environments evaluate identical package expressions regardless of subsequent upstream updates.

This distinction is important for practical classroom deployment. An outage affecting a source host such as GitHub [35] can interfere with updating a flake and can also interfere with first use when a locked input has not yet been fetched. These cases should not be conflated with failure of an already available, locked environment. Likewise, an instructor's decision to refresh a lock file can reveal renamed, removed,

or insufficiently maintained package attributes that were not present in the version used for a prior course offering.

4) Realization, Binary Caches, and Trust: Nix evaluation determines whether an expression is syntactically and semantically valid and identifies the derivations it requires. Realization obtains the corresponding store paths. During realization, Nix first attempts to substitute a pre-built store path from a configured binary cache; the public cache at `cache.nixos.org` is the set default [28, 36]. If no acceptable substitute is available, Nix can build the derivation from source, provided that its sources and build dependencies can be obtained. This fallback improves completeness but may require substantial time and local computing resources for large dependency closures.

A *substituter* is an additional binary-cache endpoint [36]. It can reduce setup time by serving store paths that are absent from the primary cache, including course-specific builds maintained by an institution. However, cache use is also a supply-chain trust decision. Nix accepts binary substitutes according to configured public signing keys; adding a third-party cache together with its key expands the set of entities authorized to provide binaries that students execute. Thus, a substitute may improve availability while increasing the trusted computing base [4, 37].

These mechanisms establish the technical basis for the operational constraints discussed in Section C. Classroom dependability relies on more than whether a `flake.nix` evaluates: locked inputs must remain obtainable when needed, package definitions must remain suitable for the course, required binaries must be available

or buildable within the instructional time budget, and any cache used to improve availability must be explicitly trusted.

C. Summary of Research Gaps

Existing literature demonstrates a clear trade-off in managing development environments in computer science courses. Centralized servers introduce administrative friction and network vulnerabilities [1, 3]. Virtual machines may introduce computational and storage loads that lower-end student machines struggle to handle [3, 26]. DevContainers deliver standardized isolation at the expense of background virtualization on non-Linux hosts, disk storage consumption, and imperative image build layers [1, 9]. Nix Flakes offer a compelling alternative that balances native application performance, fine-grained reproducibility, and unprivileged execution [8, 14], yet their functional DSL and packaging conventions impose a steep learning curve that restricts broad classroom adoption [6, 8, 16].

Although modern LLMs excel at general-purpose code synthesis [38], they exhibit low pass rates on declarative infrastructure specifications [19, 20] and elevated package hallucination rates in less-represented ecosystems [21]. While ecosystem wrapper tools such as `devenv` introduced an AI-powered `devenv generate` CLI command in version 1.4 to scaffold Nix-based development environments from existing project files, that approach required adopting the `devenv` abstraction layer rather than producing standalone, native `flake.nix` specifications. Moreover, because `devenv generate` routed all requests through a single vendor-hosted endpoint with no option for local execution or self-hosting, its codebase-scanning

CLI capability was subsequently removed following community backlash over privacy and telemetry practices that uploaded tracked repository contents to an external server [39]. Furthermore, no prior academic work has investigated combining specialized custom agent skills, local codebase and document requirement inference, real-time MCP registry retrieval, and deterministic Nix evaluation feedback to bridge the cognitive divide between natural language course requirements and functional development environments.

TABLE I
COMPARISON OF DEVELOPMENT ENVIRONMENT MANAGEMENT APPROACHES

Approach	**Advantages**	**Disadvantages**
Institution-Hosted Server	• Single environment maintained in one place where all the software used is configured [1] • Instructor and IT control over resources and access	• Requires a persistent, low-latency network or VPN [1] • Depends on central IT for setup and toolchain updates [1, 3] • Server outage affects the entire class [3]
Virtual Machines	• Strong isolation from the host [2, 26] • Complete, uniform operating system for all students [26] • Supports any guest OS and toolchain [26]	• Multi-gigabyte images to distribute and store [3, 26] • High CPU, memory, and disk overhead [1, 3] • Slow startup; hardware virtualization required [1, 3] • Imperative installation
DevContainers	• Environment described in a version-controlled specification [1] • Lighter than full VMs; shares the host kernel [1, 3] • Editor integration (e.g., VS Code) [1, 3]	• Docker daemon on macOS and Windows requires a background Linux VM [1] • Docker images may take significant amount of storage [1] • Slower file I/O on macOS and Windows [1] • Rebuilds are not guaranteed bit-for-bit reproducibility [8, 9]
Nix Flakes	• Declarative, pinned inputs via `flake.lock` [6, 8] • Native performance • Shared store avoids duplicated dependencies [11, 40] • No root privileges needed for activation [8]	• Steep learning curve of the Nix language [6, 8] • Experimental, breaking changes are possible • Weaker isolation than VMs or containers • Requires network or binary cache for first use [28]

Chapter III.

Research Questions and Proposed Architecture

A. Research Questions and Hypotheses

This thesis investigates the feasibility, reproducibility, and reliability of automated declarative environment generation for computer science education. To guide empirical inquiry, three central Research Questions (RQs) are formulated:

1. **RQ1:** Can AI automatically generate functional development environments from CS assignment requirements?
2. **RQ2:** Can those generated environments be reproduced consistently across different computing platforms?
3. **RQ3:** What are the potential benefits and practical limitations of this approach for student onboarding?

To evaluate these questions systematically, we formulate four corresponding working hypotheses. These hypotheses guide our empirical investigation into agentic flake generation, host-independent reproducibility, and student activation ergonomics:

- **H_{1a}:** Providing structured procedural scaffolding (`course-flake-generator`) and closed-loop evaluation (`nix flake check`) will enable both frontier (`Gemini 3.8 Flash`) and smaller open-weight (`GPT-OSS 120B`) models to reliably synthesize valid `flake.nix` environments, narrowing the synthesis accuracy gap between model tiers.

- **H_{1b}:** Augmenting the agent with real time data (`mcp-nixos`) will reduce package hallucinations and improve first-attempt synthesis accuracy (Pass@1) and package precision compared to relying on training data knowledge and native Nix CLI commands alone.
- **H_2:** Nix Flake environments that pass the host validation pipeline with a pinned `flake.lock` will deterministically reproduce and support coursework execution in clean, unmounted Linux containers and macOS virtual machines, with any residual failures isolated to platform-specific package availability.
- **H_3:** Single-command activation via `nix develop` will reduce procedural errors and cognitive friction compared to manual imperative installation.

B. Proposed Solution: An agent skill for AI-Assisted Nix Flake Synthesis

To bridge the gap between complex declarative configurations and educational usability, we propose a custom agent skill that specializes an autonomous coding agent in synthesizing and verifying valid `flake.nix` specifications. Rather than requiring instructors or students to master the Nix expression language, the skill delegates dependency inference, package resolution, and build verification to the user's local agent harness. Although existing wrapper ecosystems such as `devenv` introduced an AI-assisted `devenv generate` command in version 1.4, that approach requires the installation of `devenv` tool and subsequently had its CLI codebase-inference capability removed following privacy concerns over telemetry that uploaded tracked repository files to external servers [39]. In contrast, our proposed skill operates directly within the local agent harness to produce standard,

zero-wrapper `flake.nix` configurations while inspecting project repositories and course documents locally.

Operating this synthesis pipeline requires a host environment that satisfies three baseline prerequisites. First, the host system must run a POSIX-compatible environment supported by the Nix package manager: Linux (natively or within Docker containers), macOS, or Microsoft Windows via the Windows Subsystem for Linux (WSL2). Second, Nix must be installed with experimental Flake support enabled. When using the official Nix installer, users must explicitly enable flakes by declaring `experimental-features = nix-command flakes` in either the user-level (`~/.config/nix/nix.conf`) or system-wide (`/etc/nix/nix.conf`) configuration file; alternatively, the Determinate Systems Nix installer enables these features by default without manual intervention. Third, a compatible agent harness must be installed to load the skill package and orchestrate tool execution. Because Agent Skills follow an open, modular specification consisting of structured Markdown instructions (`SKILL.md`), reference templates, and helper scripts [23, 24], our skill is portable across any agent harness that supports the standard skill directory convention and local shell tool invocation.

Operating through the instructor's local agent harness, the custom agent skill guides an assignment-to-flake workflow (illustrated at a high level in Figure 1) across six sequential stages. Each stage enforces explicit validation boundaries to ensure that only verified configurations progress toward student deployment:

1. **Assignment document:** The instructor supplies existing coursework materials (such as an assignment handout in PDF, DOCX, HTML, or Markdown format, natural language instructions, or an unconfigured starter codebase) without needing to manually compose Nix expressions.

2. **Requirement extraction:** The agent extracts required compilers, language runtimes, build tools, and system libraries from the provided materials. This capability is driven by the custom skill's bundled scripts: a document extraction script to parse unstructured assignment handouts, alongside a project detector that scans repository files and manifests for declared dependencies.

3. **Flake generation:** Guided by the `course-flake-generator` skill's multi-architecture templates and reference heuristics, the agent synthesizes a declarative `flake.nix` specification. To resolve required package attributes deterministically, the skill primarily queries the `mcp-nixos` server against real-time Nixpkgs search indices, falling back to native Nix CLI inspection commands when the MCP server is not present.

4. **Validation:** The candidate `flake.nix` is evaluated through closed-loop verification, executing `nix flake check` and ephemeral shell realization checks to detect syntax errors or missing executables. If failures occur, compiler diagnostics are returned to the agent harness for automated self-correction until validation passes and generates a pinned `flake.lock`.

5. **Instructor approval:** The instructor reviews the synthesized `flake.nix` and cryptographically pinned `flake.lock` file and commits them to the assignment repository, establishing a verified, frozen dependency closure for the course.

6. **Student activation:** Students clone the assignment repository onto their personal devices and run `nix develop` to enter the reproducible development shell, avoiding manual installation troubleshooting and working directly on the assignment.

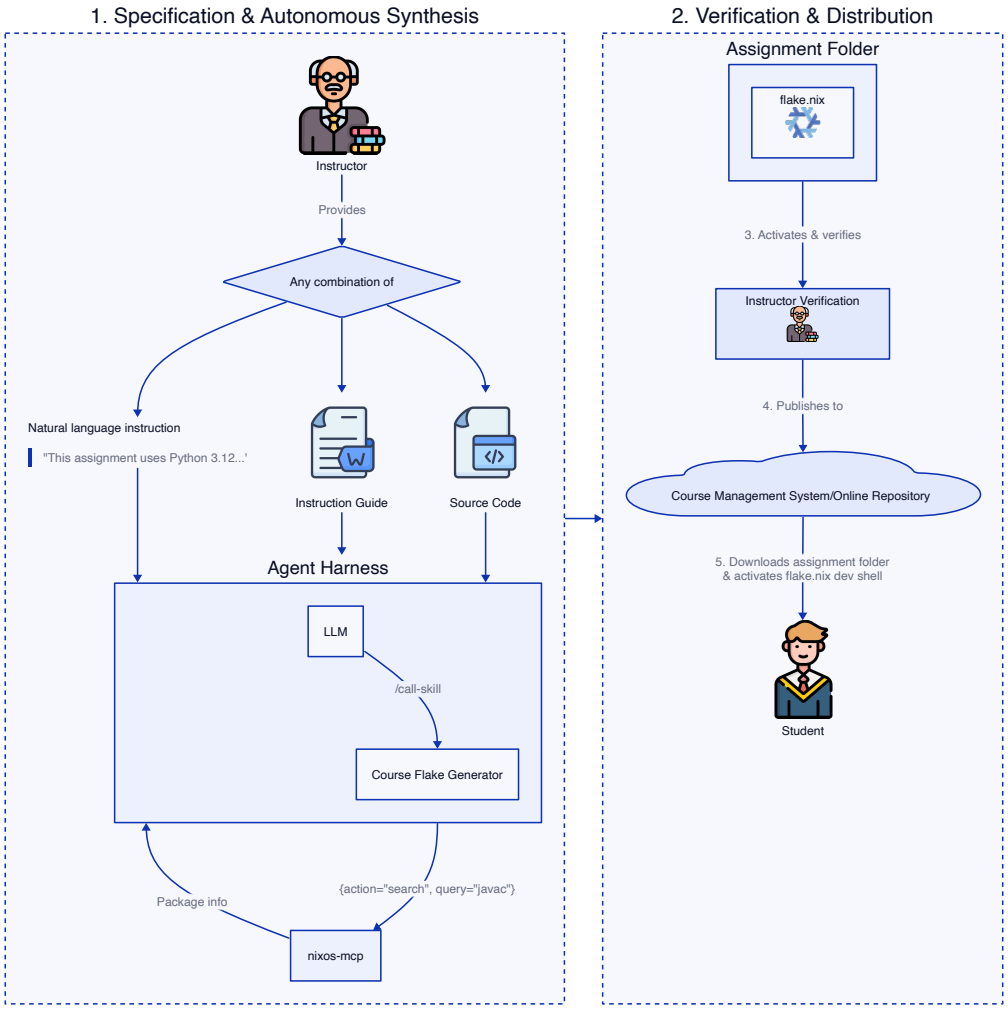


Fig. 1. End-to-End Architecture of the Autonomous `flake.nix` Synthesis Pipeline
Icons created by Magnific - Flaticon

Chapter IV.

Experimental Methodology and Test Setup

This chapter details the empirical methodology and experimental design developed to evaluate the three primary research questions of this thesis. Section A outlines the agentic flake generation test protocol, including dataset curation, and the multi-stage Pass@k validation metric. Section B formalizes the clean-environment replay designed to assess host-independent reproducibility of the generated flake across Linux containers and macOS virtual machines. Section C describes the user study protocol examining student setup time delta and cognitive ergonomics when onboarding with `nix develop` versus imperative instructions (RQ3). Finally, Section D consolidates the experimental testbed infrastructure, documenting host hardware specifications and guest virtualization environments.

A. Agentic Flake Synthesis Evaluation Protocol

To investigate whether modern autonomous agents can reliably synthesize declarative Nix development environments from informal natural language requirements (RQ1), we established a benchmark evaluation protocol. This protocol couples structured curriculum task suites with an automated verification pipeline to measure synthesis accuracy, dependency resolution, and runtime execution determinism.

1) Benchmark Dataset Curation and Evaluation Suites: Because standardized benchmark datasets for declarative Nix environments did not previously exist, we

constructed an evaluation benchmark representing standard computer science coursework requirements across three distinct instructional modalities. These modalities span informal prompt instructions, complete student project repositories, and unformatted assignment handouts to capture various inputs:

1. **Natural Language Specification Prompts:** Curated prompts representing informal assignments provided by instructors across two curriculum tracks:
    - *Software Development Track:* Assignments focusing on systems programming, compiler toolchains, and web engineering. These tasks require native compilers, build systems, and runtime interpreters, including C/C++ (GCC, Clang, CMake, GNU Make), Rust (Cargo, `rustc`), and full-stack environments (Python, Node.js).
    - *Data Science Track:* Coursework requiring complex scientific computing stacks, numeric routines, and specialized packages, including Python (NumPy, SciPy, Pandas, PyTorch), R statistical environments, and Jupyter notebook kernels.

    The prompt benchmark is serialized as a `jsonl` file, where each record specifies the curriculum track, the natural language prompt, and the asserted dependency list (Listing 3).

2. **Existing Coursework Codebases:** A collection of project repositories collected from computer science coursework spanning artificial intelligence assignments, object-oriented Java laboratories, data structures projects, mobile app development, neural network coursework, and network programming. For

these tasks, the agent cannot rely on explicit textual package lists; instead, the skill instructs the agent to run `detect-project-requirements.py` to inspect file contents, repository structures, and build manifests (such as `pom.xml`, `build.gradle`, and `pyproject.toml`) to infer necessary runtimes.

3. **Instructional Course Documents:** A collection of course documents and laboratory setup guides formatted as PDF, Microsoft Word (`.docx`), HTML, and Markdown files. To process these unstructured documents without manual keyword pre-mapping, the skill executes `extract-document-content.py` to parse layout structures into clean text for agent requirement extraction.

To establish an objective evaluation standard across all three modalities, each task is paired with an evaluation schema defining ground-truth requirements. While synthetic prompt benchmarks are serialized as a unified `jsonl` file, each codebase directory and instructional document is paired with an individual companion metadata `.json` file (Listing 3). This metadata serves as the authoritative ground truth, manually verified by a human evaluator to establish the passing criteria for the assignment environment.

```
{
  "title": "AI Assignments - Artificial Intelligence Course",
  "track": "computer science",
  "instruction": "Set up a Nix flake development environment...",
  "dependency_list": [
    "python3",
    "jupyter",
    "git"
  ]
}
```

Listing 3. Evaluation Metadata Format and Ground-Truth Schema

In this schema, `track` classifies the curriculum domain, `title` and `instruction` provide contextual task descriptions, and `dependency_list` designates the ground-truth executables that must resolve inside the ephemeral `nix develop` shell. The automated evaluation harness evaluates these human-verified assertions to determine whether candidate flakes satisfy the Pass@k criteria and to compute package precision and over-provisioning metrics.

2) Agent Harness, Model Configurations, and Tooling Conditions: The experimental synthesis harness was implemented using the Antigravity CLI harness. This harness coordinates prompt delivery, tool routing, and shell execution streaming.

We also wanted to assess how model architecture and reasoning scale influence Nix expression synthesis, experiments were conducted across two language models: Google Gemini 3.8 Flash and GPT-OSS 120B, both configured with medium reasoning effort. Evaluating models across different parameter scales allows us to assess whether smaller, cost-effective models can synthesize functionally equivalent environments when equipped with appropriate contextual scaffolding.

To guide the agent and eliminate erratic zero-shot behavior, each agent session integrates our specialized `course-flake-generator` skill (introduced in Section B). The internal filetree structure of the custom agent skill follows standardized agentic skill packaging conventions:

```
.
├── README.md
├── SKILL.md
├── assets
│   ├── flake-devshell.template.nix
```

```
│   └── mcp-servers.template.json
├── references
│   ├── flake-patterns.md
│   ├── inferring-requirements.md
│   └── mcp-nixos.md
└── scripts
    ├── detect-project-requirements.py
    ├── extract-document-content.py
    └── validate-flake.py
```

Listing 4. Custom Skill Filetree Structure

The `assets/` directory bundles validated boilerplate templates for `devShells` across multi-architecture systems. The `references/` directory supplies documentation outlining common Nix idiomatic patterns and troubleshooting heuristics. The `scripts/` directory stores Python scripts to perform deterministic actions such as extracting text from user submitted documents. According to Ling et al. [24], providing modular templates and clear reference bounds significantly improves behavioral consistency and task adherence in autonomous agents.

```
{
  description = "{{DESCRIPTION}}";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.11";
    flake-utils.url = "github:numtide/flake-utils";
  };

  outputs =
    {
      nixpkgs,
      flake-utils,
      ...
    }:
    flake-utils.lib.eachDefaultSystem (
      system:
      let
        pkgs = import nixpkgs {
          inherit system;
```

```
          config.allowUnfree = true;
        };
      in
      {
        devShells.default = pkgs.mkShell {
          packages = with pkgs; [
            {{PACKAGES}}
          ];

          {{ENV_VARS}}

          shellHook = ''
            {{SHELL_HOOK}}
          '';
        };

        formatter = pkgs.nixfmt-rfc-style;
      }
    );
}
```

Listing 5. assets/flake-devshell.template.md template

Additionally, we evaluated two distinct package discovery conditions to measure the impact of external tool grounding. Comparing these configurations isolates whether autonomous agents strictly require external registry access or can reliably synthesize environments using command-line tools alone:

1. **Live Registry Retrieval via MCP-NixOS:** The agent queries the `mcp-nixos` server, which communicates directly with the Elasticsearch index powering `search.nixos.org`. Because Nixpkgs hosts over 139,008 packages across 1,044 package sets, real-time index querying protects against package hallucinations, deprecated attribute paths, and naming drift [21, 31].

2. **Native Nix CLI Discovery:** In this condition, the MCP server is withheld from the agent, forcing it to rely exclusively on parametric pre-training weights and native Nix command-line inspection tools (such as `nix search` and `nix-env -qaP`).

An automated Python test runner was developed to execute test suites, capture agent tool call telemetry, stream shell outputs, and record execution success metrics. The runner logs detailed traces for every synthesis attempt to facilitate post-hoc telemetry analysis.

Each benchmark invocation creates a timestamped run directory storing evaluation metrics, aggregated telemetry, and isolated workspace artifacts for every evaluated task and candidate sample. The resulting directory structure is organized as shown in Listing 6.

```
20261028_143024/
├── benchmark_report.md
├── benchmark_summary.json
├── mcp_tool_frequency.csv
├── mcp_tool_log.csv
├── run_command_base_frequency.csv
├── run_command_frequency.csv
├── run_command_log.csv
├── task_00
│   ├── sample_00
│   │   ├── agent_response.md
│   │   ├── transcript.jsonl
│   │   └── trial_meta.json
│   ├── sample_01
│   │   ├── agent_response.md
│   │   ├── transcript.jsonl
│   │   └── trial_meta.json
│   └── sample_02
│       ├── agent_response.md
│       ├── transcript.jsonl
│       └── trial_meta.json
├── tool_call_frequency.csv
```

```
├── tool_call_frequency_by_task.csv
└── tool_call_frequency_by_trial.csv
```

Listing 6. Benchmark Execution Output Directory Structure

The artifacts generated within each benchmark run serve distinct functions for performance aggregation, behavioral auditing, and trial verification. These files capture telemetry across multiple levels of granularity, enabling reproducible post-hoc analysis:

- `benchmark_summary.json`: Aggregates Pass@k metrics, execution latency, token consumption, and dependency bloat scores across all trials. It serves as the primary machine-readable dataset for statistical analysis.
- `benchmark_report.md`: Summarizes benchmark results, per-task score tables, and execution parameters in Markdown format. It provides an immediate overview of benchmark outcomes and links to individual agent sessions.
- `tool_call_frequency.csv`: Records the overall frequency, trial prevalence, and average invocations of every agent tool call. It quantifies how agent tool usage shifts between different model and grounding conditions.
- `tool_call_frequency_by_task.csv`: Cross-tabulates tool calls across individual coursework assignments, revealing which problem domains introduced heavier tool utilization.
- `tool_call_frequency_by_trial.csv`: Maps tool invocations at the level of individual trial samples alongside validation success and duration, facilitating fine-grained correlation between tool call patterns and task outcomes.

- `run_command_frequency.csv` and `run_command_base_frequency.csv`: Tabulate exact and base shell commands executed through the agent harness. They reveal the CLI inspection patterns and build utilities invoked during synthesis.
- `run_command_log.csv`: Logs a complete, timestamped chronological trace of every shell command executed, its working directory, and associated metadata.
- `mcp_tool_frequency.csv` and `mcp_tool_log.csv`: Capture all registry search queries, filter arguments, and actions dispatched to the mcp server, if applicable.
- `agent_response.md`: Preserves the final conversational completion message and instructions returned by the agent. It allows evaluators to verify the clarity and correctness of the agent's onboarding guidance.
- `flake.lock`: Pins exact cryptographic source hashes and input commits generated during local validation. It guarantees reproducible, deterministic evaluation across clean guest environments.
- `flake.nix`: Defines the declarative `devShell` environment and package inputs synthesized by the agent. It is the primary artifact tested against the three-stage validation pipeline.
- `jupyter_install.html`: Contains the localized assignment specification or handout provided to the agent in document-grounded tasks. Isolating this input guarantees a self-contained trial workspace.

- `raw_output.log`: Captures unbuffered standard output and standard error streams from the agent harness execution. It assists with post-mortem debugging whenever an execution hangs or crashes unexpectedly.
- `transcript.jsonl`: Stores the comprehensive turn-by-turn event stream, including internal reasoning traces, tool parameters, and raw tool responses. It provides full behavioral provenance for qualitative evaluation.
- `trial_meta.json`: Records structured trial execution metrics, including pass/fail status, validation stages completed, latency, token usage, and dependency precision. It supplies the ground-truth values used to calculate Pass@k and bloat statistics.

3) Three-Stage Validation Pipeline and Pass@K Formulation: Synthesizing a declarative `flake.nix` file requires navigating a functional domain-specific language where surface syntactic correctness does not guarantee runtime deployability [20]. To rigorously verify that generated environments are functional, every candidate flake must satisfy a three-stage automated validation pipeline:

1. **Syntax and Evaluation Validity:** The command `nix flake check` evaluates the generated `flake.nix` with lock-file writes disabled, ensuring that the expression parses cleanly without syntax errors, missing attributes, or invalid type signatures.

2. **Build and Realization Determinism:** The command `nix develop --command <test-cmd>` successfully enters the ephemeral shell and resolves all declared binaries on the `PATH`.

3. **Functional Task Execution:** The assignment source code builds and passes its unit test suite inside the synthesized `devShell`, verifying that compiler toolchains and runtime headers interact correctly with project source files.

To quantify synthesis reliability, we adapted the Pass@k formulation from code generation literature [38]. Pass@k measures the probability that at least one of k generated candidate environments passes all validation stages, where we generate $n \geq k$ samples per task. For our evaluation, we set $k = 3$ and $n = 3$.

This formulation calculates the probability that an instructor obtains at least one working environment without having to evaluate every trial sequentially. Specifically, we calculate Pass@k using the unbiased estimator introduced by Chen et al. [38]:

$$\text{pass@}k := \mathop{\mathrm{E}}_{\text{Problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right]$$

where n is the total number of generated candidate samples per task and c is the number of samples that successfully pass all three validation stages.

B. Clean-Environment Reproducibility Protocol

RQ2 evaluates whether a successful local validation actually transfers to a fresh environment that has none of the authoring host's files, shell configuration, or prior evaluation state. The inputs contain only RQ1 trial artifacts that passed the three-stage local verifier and include both a generated `flake.nix` and its `flake.lock`. This design evaluates the reproducibility of the generated flake specification.

On a local development machine, an existing warmed Nix store or globally installed host library may mask missing dependency declarations, creating a false impression of reproducibility. To isolate the environment from authoring host pollution, our protocol mandates that candidate flakes be tested in fresh, ephemeral guest environments where no host directories or store paths are mounted.

The harness streams a compressed copy of the trial into the guest environment and extracts it into guest-local temporary storage; no project directory or host Nix store is mounted. The guest instance is discarded when the stage finishes, so no writable state is shared across stages or trials. Network access remains available so that a first-use environment can retrieve its pinned inputs and binary-cache artifacts, and availability failures are recorded separately from configuration failures.

1) Guest Replay Pipeline and Failure Taxonomy: The guest-side verifier executes four sequential stages on each RQ1-passing artifact. These stages mirror the host verification harness while preventing unstated host assumptions from masquerading as portable configurations:

1. **Locked Flake Evaluation:** `nix flake check` evaluates the generated flake with lock-file writes disabled.

2. **Development-Shell Activation:** `nix develop --command true` verifies that the development shell can be activated.

3. **Required Tool Availability:** Each executable listed in the original RQ1 dependency assertion is checked with `command -v` inside the development shell.

4. **Headless Task Execution:** A curated per-task command set verifies applicable language runtimes, builds, dependency synchronization, or test commands. Graphical editor workflows, interactive credentials, and persistent database-service administration are excluded because they cannot be assessed consistently by a headless guest.

The principal metric is the base OS reproduction rate: the proportion of trials that complete every applicable container-side stage. Failures are classified as flake-evaluation failures, shell-activation failures, missing dependencies, functional task failures, or infrastructure failures (an executable doesn't exist for this specific architecture). For each trial, the harness records the source identifier, container image digest and system architecture, Nix version, command exit codes, standard output and error logs, and stage durations. These artifacts make it possible to distinguish a defective generated environment from a transient container or upstream package-source failure.

C. Student Onboarding Ergonomics Protocol

RQ3 evaluates the practical ergonomics and setup time delta of Nix Flakes from a student perspective compared to traditional manual installation. To assess human ergonomics and onboarding friction, we conducted a controlled user study involving two undergraduate students and a graduate computer science student. Each participant was tasked with configuring a course development environment using two contrasting methods: traditional imperative instructions from a course

document, and a `flake.nix` file that was generated from the above instruction guide using our custom agent skill.

Each trial was executed with 2 clones of a vanilla macOS virtual machine (1 without nix and one with) from Cirrus Labs, invoked via `tart run --dir=host:./<student-A-dev>` where `--dir` is the current directory of the AI generated flake. In the macOS VM, this becomes a shared volume, automatically providing the VM the required files.

We recorded total setup completion time, the time delta between imperative installation and declarative shell activation, the student's personal assessment of their preferred workflow, and qualitative observations regarding cognitive load. In addition, we tracked procedural errors, including skipped setup steps and terminal configuration issues, to determine how each method influenced setup friction.

We evaluated three progressively complex coursework environments to capture how setup time and procedural friction scale with dependency complexity. The first task required installing a single compiled language toolchain consisting of the Rust compiler and Cargo build system. The second task involved provisioning a data science environment with a Python runtime and the Jupyter Notebook package. Finally, the third task required configuring a multi-component full-stack web development environment integrating multiple language runtimes, package managers, and database utilities.

D. Experimental Testbed and System Infrastructure

To ensure consistent, reproducible execution across all evaluation stages, the experimental testbed relies on a macOS host machine running an isolated macOS virtual machine. Table II summarizes the hardware and system environment of the primary host machine used for synthesis orchestration and local validation.

TABLE II
HOST MACHINE HARDWARE AND SYSTEM ENVIRONMENT

Attribute	**Specification**
Host Model	Apple MacBook Air
Processor	Apple M5 (10 cores: 4 Performance, 6 Efficiency)
Architecture	`arm64` (`aarch64`)
RAM	24 GB Unified Memory
Operating System	macOS 27.0.1
Nix Version	Nix 2.24+ (Flakes enabled)

For guest-level replay and ergonomics testing, the testbed configures two distinct virtualization tiers. These tiers decouple evaluation from the authoring host while replicating the primary desktop platforms used by students:

- **Linux Replay Containers:** To simulate standard Linux and Windows Subsystem for Linux (WSL) environments, validation stages are replayed inside unmounted Docker containers running official Nix images. Each trial executes in an isolated container lifecycle without shared filesystem volumes or pre-warmed host store paths.
- **macOS Tart Virtual Machines:** To evaluate macOS and Apple Silicon guest environments, we utilize Tart, an open-source virtualization framework optimized for Apple Silicon hypervisors. We obtained a clean image of macOS

Golden Bridge via `ghcr.io/cirruslabs/macos-golden-gate-base:latest` `macos-27-base`.

Chapter V.

Empirical Results and Analysis

A. Flake Synthesis Accuracy and Model Comparison

Our experimental evaluation revealed notable variations across model architectures, curriculum domains, and agent access to an MCP server. In our initial evaluations conducted without the custom agent skill, general-purpose LLMs exhibited a strong tendency toward imperative fallbacks. Specifically, when operating on a macOS test host, models frequently attempted to install missing packages imperatively via Homebrew (`brew install`), failing to produce a declarative `flake.nix` file entirely, despite explicitly prompting the model.

With the introduction of the `course-flake-generator` skill, the model behaved more consistently. The agent successfully adhered to the declarative flake pattern for over 85% of standard software development prompts. However, curriculum domain complexity strongly influenced outcomes. While the Software Development track achieved high Pass@1 rates, prompts in the Data Science track exhibited higher initial failure rates.

Under the Pass@3 metric, Gemini 3.8 Flash succeeded on at least one attempt for every assignment prompt in the benchmark (100.00% Pass@3), whereas GPT-OSS 120B achieved 78.57% Pass@3 with MCP access and 85.71% without MCP access. These results partially support Hypothesis $\mathbf{H_{1a}}$: structured procedural scaffolding and closed-loop validation enabled both the frontier model and the

smaller open-weight model to synthesize functional environments across the majority of coursework prompts, though a performance gap persisted between model tiers. As detailed in Section B, this residual gap stemmed primarily from differences in tool utilization: GPT-OSS 120B made significantly fewer calls to the MCP server and relied more heavily and confidently on its training data without calling the native Nix CLI search command, leading to higher initial hallucination rates for less common package attributes.

Table III summarizes the overall synthesis accuracy across both models with and without access to the `mcp-nixos` server, Table IV reports mean trial duration and token consumption, and Table V presents package precision and the over-provisioning (bloat) ratio across passing trials. To evaluate partial success, the evaluator extracts all package identifiers declared in `packages`, `buildInputs`, and `nativeBuildInputs` (exempting standard Nix formatters such as `nixfmt` or `alejandra`) and maps them against the expected CLI tools for each task. *Package precision* measures the fraction of declared packages that directly satisfy an expected tool requirement (penalizing unrequested, extraneous packages), whereas the *over-provisioning ratio* (bloat) computes the ratio of total declared packages to the minimum number of required packages needed to cover the expected tools, capturing both extraneous additions and redundant package declarations. Evaluating these metrics is critical because language models frequently introduce unrequested dependencies and extraneous configurations when generating declarative specifications [19–21].

TABLE III
FLAKE SYNTHESIS ACCURACY ACROSS MODELS AND MCP CONDITIONS

Model	**Condition**	**Passed** (c/N)	**Pass@1**	**Pass@3**
Gemini 3.8 Flash	w/ MCP	40/42	95.24%	100.00%
Gemini 3.8 Flash	w/o MCP	41/42	97.62%	100.00%
GPT-OSS 120B	w/ MCP	31/42	73.81%	78.57%
GPT-OSS 120B*	w/o MCP	30/42	71.43%	85.71%

TABLE IV
MEAN TRIAL DURATION AND TOKEN CONSUMPTION ACROSS MODELS AND MCP CONDITIONS

Model	**Condition**	**Mean Duration**	**Mean Tokens**
Gemini 3.8 Flash	w/ MCP	238.41 s	221,295
Gemini 3.8 Flash	w/o MCP	233.85 s	235,596
GPT-OSS 120B	w/ MCP	180.03 s	26,974
GPT-OSS 120B	w/o MCP	89.51 s	38,225

TABLE V
PACKAGE PRECISION AND OVER-PROVISIONING RATIO ACROSS PASSING TRIALS

Model	**Condition**	**Package Precision**	**Over-Provisioning Ratio**
Gemini 3.8 Flash	w/ MCP	93.3%	1.65×
Gemini 3.8 Flash	w/o MCP	91.2%	1.87×
GPT-OSS 120B	w/ MCP	93.5%	1.39×
GPT-OSS 120B	w/o MCP	92.8%	1.43×

1) Codebase-Inferred and Document-Based Flake Synthesis: To evaluate whether the `course-flake-generator` skill scales beyond synthetic natural language prompts to real-world educational materials, we evaluated its accuracy across six existing coursework repositories and nine instructional setup documents described in Subsection 1. Both Gemini 3.8 Flash and GPT-OSS 120B were evaluated across these modalities under identical skill scaffolding and closed-loop verification conditions. Table VI presents the comparative synthesis performance and execution telemetry on existing coursework repositories (6 tasks, 18 total trials), while Table VII reports the corresponding evaluation metrics on instructional setup documents (9 tasks, 27 total trials).

TABLE VI
SYNTHESIS PERFORMANCE ON EXISTING COURSEWORK CODEBASES

Model	**Passed (c/N)**	**Pass@1**	**Pass@3**	**Precision**	**Over-Prov.**
Gemini 3.8 Flash	18/18	100.00%	100.00%	42.2%	3.73×
GPT-OSS 120B	5/18	27.78%	66.67%	87.1%	1.80×

TABLE VII
SYNTHESIS PERFORMANCE ON INSTRUCTIONAL COURSE DOCUMENTS

Model	**Passed (c/N)**	**Pass@1**	**Pass@3**	**Precision**	**Over-Prov.**
Gemini 3.8 Flash	27/27	100.00%	100.00%	49.6%	2.90×
GPT-OSS 120B	17/27	62.96%	77.78%	58.6%	2.89×

As shown in Table VI and Table VII, Gemini 3.8 Flash achieved a flawless 100.00% Pass@1 and Pass@3 across all 18 codebase trials and all 27 document trials. In contrast, GPT-OSS 120B attained 27.78% Pass@1 (5/18) and 66.67% Pass@3 (4/6 tasks) on existing codebases, and 62.96% Pass@1 (17/27) and 77.78% Pass@3 (7/9 tasks) on instructional course documents. A qualitative review of each trial's

execution event stream (`transcript.jsonl`) and trial metadata revealed distinct failure modalities, behavioral patterns, and architectural differences between the two models:

1. **Multi-Module Incompletion:** On coursework repositories with nested submodules, Gemini 3.8 Flash systematically iterated across every directory or authored module-specific flakes as required. In contrast, GPT-OSS 120B suffered from premature task termination. In `COSC236_Labs` (Trial 2), the model authored flakes for `Lab1` and `Lab2`, emitted a conversational status message listing "Next steps", and concluded execution after only 26.8 s, omitting Labs 3 through 10. Similarly, in Trial 3 of the same task, it generated flakes for Labs 1 through 9 but skipped `Lab10`. In `COSC336-Assignments` (Trial 1), it authored flakes for Assignments 1 through 6 but failed to process Assignments 7 and 8.

2. **Package Hallucination:** When analyzing Python projects without explicit requirements manifests (`AI-Assignments`, Trial 3), GPT-OSS 120B fell back to treating language-level libraries as top-level Nixpkgs derivations, declaring `pkgs.pandas` and `pkgs.matplotlib` in `packages` rather than constructing an idiomatic `python3.withPackages` environment. This triggered immediate Nix evaluation failure: *"error: attribute 'pandas' in selection path 'pkgs.pandas' not found. Did you mean pandoc?"*. In contrast, Gemini 3.8 Flash leveraged the skill's reference guides to use Python package sets using `python3.withPackages (ps: with ps; [ pandas matplotlib ])`.

3. **Indiscriminate Over-Provisioning and Activation Timeouts:** In `COSC336-Assignments` (Trial 2), GPT-OSS 120B observed LaTeX assignment documentation alongside Java source files and added `pkgs.texliveFull` to `nativeBuildInputs`. Because `texliveFull` is approximately a 7 GB package, downloading and substituting store paths exceeded the test harness's 60-second activation threshold, triggering an activation timeout failure.

4. **Empty Output Errors:** In several document-grounded trials (such as Task 4 Trial 3 and Task 8 Trial 1), GPT-OSS 120B generated empty conversational responses between background tool calls, triggering harness runtime errors (*"model output error: model output must contain either output text or tool calls"*) and exiting prior to generating `flake.nix`.

5. **Naming Drift on Deprecated Packages:** On the database setup guide (`Itec 451 Mcguire1`), all three GPT-OSS 120B trials failed because the model repeatedly declared `pkgs.mysql`, which has been removed and replaced by `mariadb` and versioned attributes (`mysql80`) in Nixpkgs 24.11. Because the model withheld MCP search calls and relied exclusively on parametric weights, it was unable to detect and recover from attribute renaming.

6. **Nix Identifier Syntax Violations:** In the full-stack Elixir task (`Fullstack Elixir Sqlite Evaluation`, Trial 2), GPT-OSS 120B attempted to declare Node.js via `nodejs-20_x`. In Nix syntax, unquoted hyphens in attribute

identifiers are evaluated as subtraction operators, causing Nix to fail with *"error: undefined variable 'nodejs-20_x'"*.

Interestingly, on passing trials, GPT-OSS 120B achieved a higher package precision (87.1% vs. 42.2%) and a lower over-provisioning ratio (1.80× vs. 3.73×) than Gemini 3.8 Flash on existing codebases. Transcript inspection explains this apparent paradox: when GPT-OSS 120B succeeded, it generated strictly minimalist flakes containing only the core compilers explicitly tested by the verifier (such as `openjdk17` or `pythonEnv`). Conversely, Gemini 3.8 Flash proactively inferred and provisioned comprehensive developer tooling suites detected within project files, including version control (`git`), build automation tools (`gradle`, `just`), and data science libraries (`torch`, `torchvision`, `scikit-learn`, and Jupyter kernels). While these auxiliary packages were penalized as extraneous under the strict CLI assertion baseline, they reflect a more complete, production-ready developer environment for students.

This tension between minimal specification adherence and unprompted over-generation is well documented in automated infrastructure synthesis. In their benchmark evaluation on Infrastructure as Code, Kon et al. [19] demonstrated that over 50% of configurations that successfully passed compiler checks violated user intent specifications by provisioning unrequested, extraneous cloud resources and services. Similarly, Zhang et al. [20] observed that fewer than 26% of synthesized deployment templates adhered to user-intended resource and attribute boundaries without inserting self-defined properties or unneeded substitutions. In our

educational setting, however, qualitative analysis reveals an important distinction: whereas prior benchmarks observed unprompted resources arising from model hallucination or prompt divergence, Gemini 3.8 Flash's over-provisioning stemmed from proactive contextual inference. By scanning repository manifests for unstated but practical workflow requirements, the agent prioritized classroom deployability over minimal ground-truth precision.

Finally, the execution telemetry reported in Tables VI and VII reveals a substantial divergence in execution characteristics. GPT-OSS 120B operated significantly faster than Gemini 3.8 Flash (averaging 77.31 s vs. 324.58 s on codebases, and 48.79 s vs. 232.38 s on documents) and consumed nearly an order of magnitude fewer tokens (38,459 vs. 261,714 on codebases, and 30,873 vs. 202,707 on documents).

B. Tool Invocation and Telemetry Analysis

The tool-call logs were analyzed to determine how the agent resolved package and command uncertainty while constructing a flake. This analysis complements the pass/fail synthesis metric: a successful flake does not reveal whether the agent relied on parametric knowledge, live package retrieval, or native command-line inspection to reach the result. MCP-server availability was controlled by the experimental harness; it was not a tool-selection decision made by the model.

1) Native Nix CLI Discovery versus MCP Retrieval: Our baseline experiments evaluated whether an agent strictly requires the `mcp-nixos` server for successful synthesis. In our testing script, we added a flag to run a condition in which the MCP

server was withheld from the model. Even without MCP access, the model produced valid environments across the benchmark by combining native Nix CLI discovery with its parametric knowledge of the Nixpkgs derivation schema. Guided by the skill's procedural directive to verify that every resolved package attribute evaluates cleanly prior to finalizing `flake.nix`, the agent followed a progressive resolution workflow across the trial transcripts.

First, when resolving a target requirement from the prompt, the agent invoked `nix search` (e.g., `nix search github:NixOS/nixpkgs/nixos-24.11 dotnet-sdk`) or `nix-env -qaP` to identify candidate attribute names. Listing 7 illustrates representative output returned by `nix search` during package discovery. In addition to attribute paths and version numbers, the command supplies concise package descriptions that assisted the agent in disambiguating similar tools and identifying the appropriate derivation.

```
nix search nixpkgs ripgrep
* legacyPackages.aarch64-darwin.ripgrep (15.2.0)
  Utility that combines the usability of The Silver Searcher with the raw speed of grep

* legacyPackages.aarch64-darwin.repgrep (0.17.1)
  Interactive replacer for ripgrep that makes it easy to find and replace across files on the
    command line

* legacyPackages.aarch64-darwin.ripgrep-all (0.10.10)
  Ripgrep, but also search in PDFs, E-Books, Office documents, zip, tar.gz, and more

* legacyPackages.aarch64-darwin.ripwire (0.6.8)
  The ripgrep of AI context: a zero-dependency C++23 CLI + MCP server giving coding agents a
    ranked, deterministic map of any repo, blast radius, tests-to-run and quality deltas.
    Signatures at 80% fewer bytes than bodies; ~5% of a grep-and-read pass's tokens
```

Listing 7. Example `nix search` Output for Package Discovery

Second, because Nixpkgs frequently exposes both unversioned default aliases and version-pinned attributes (such as `dotnet-sdk` alongside `dotnet-sdk_8` and `dotnet-sdk_9`), the agent executed targeted `nix eval` expressions to disambiguate candidates before writing the flake. Specifically, even though the skill documentation did not supply a cheat sheet of derivation attributes, the model leveraged its pre-trained knowledge of Nix idioms to query `.name` and `.version` (e.g., `nix eval github:NixOS/nixpkgs/nixos-24.11#dotnet-sdk.name` and `nix eval --expr 'with import <nixpkgs> {}; openjdk17.name'`) to confirm concrete upstream versions; `.passthru.home` and `.outPath` to resolve store paths for environment variables such as `JAVA_HOME` and `DOTNET_ROOT`; `builtins.attrNames` (e.g., `builtins.attrNames pkgs.vscode-extensions`) to enumerate nested extension sets and test `vscode-with-extensions.override` expressions; and `.meta.platforms`, `.meta.broken`, and `.meta.license` to diagnose macOS (`aarch64-darwin`) compatibility for packages such as `rstudio` and `mysql-workbench`. In this benchmark, the Nix CLI therefore served as an effective, self-describing retrieval and verification interface when the MCP layer was unavailable.

Our findings align with observations by Forment et al. [18], indicating that the impact of external tool scaffolding on token efficiency depends heavily on harness implementation and task complexity. In our telemetry, access to the `mcp-nixos` server revealed a clear trade-off: external tool grounding incurred a minor token

overhead on simple tasks, yet prevented runaway troubleshooting loops and token inflation on complex environments.

For tasks involving standard packages with unambiguous derivation names (such as `python312`, `uv`, `go`, or `openjdk`), providing MCP server access resulted in higher overall token consumption. This overhead stemmed from two primary factors. First, supplying the MCP tool schema injected an additional 400 to 450 prompt tokens into the context window on every agent step. Second, the agent frequently initiated proactive exploratory queries (executing 2 to 4 MCP search queries for unambiguous tools like `python312` and `uv`) prior to drafting `flake.nix`. In contrast, when MCP access was withheld, the agent relied directly on its training data to synthesize the flake in fewer turns without exploratory retrieval overhead.

Conversely, for complex tasks requiring wrapper derivations, ambiguous attributes, or platform-specific builds (such as `gcc`, `cmake`, `rustup`, `R`, and `rstudio` on macOS), token consumption exploded in the absence of an MCP server. In Task 10 (`R`), the agent without MCP access executed numerous `nix shell`, `nix search`, and `grep` commands to determine whether `R` was an isolated derivation or required specialized wrapper scripts, increasing mean token consumption from 204,402 to 250,211 tokens (+22.4%). This divergence escalated dramatically in Task 11 (`RStudio`): in Trial 1 of the unassisted condition, the agent spiraled into an extensive troubleshooting loop lasting 133 steps, 92 shell commands, and 508,739 tokens. Without structured schema retrieval, the model attempted exhaustive `nix search` queries, evaluated raw Nix store derivations, inspected

`/nix/store/.../wrapper-rstudio.nix` via `cat`, and even reverted to imperative Homebrew invocations (`brew install --cask rstudio`) while parsing macOS application bundle crash logs before finally realizing a functional flake. Under such conditions, the live metadata exposed by `mcp-nixos` acts as an indispensable constraint boundary that halts combinatorial shell exploration.

This result should not be interpreted as evidence that MCP retrieval is generally unnecessary. The telemetry records which tools the agent used when MCP access was withheld; it does not establish whether a successful derivation originated from parametric memory, shell command output, or an interactive combination of both. The `mcp-nixos` server is particularly advantageous for rapidly evolving packages or tracking unstable channels where parametric knowledge has decayed. Under those conditions, the structured, live metadata exposed by `mcp-nixos` remains an essential mechanism for mitigating package hallucination and configuration drift [21, 31]. The present benchmark instead bounds the claim: for established core toolchains, an agent equipped with procedural scaffolding can construct a working flake using native CLI facilities alone.

When the `mcp-nixos` server was provided, the two models exhibited a stark divergence in tool adoption, as summarized in Table VIII and illustrated across individual trials in Figure 2. Gemini 3.8 Flash invoked `call_mcp_tool` 136 times across all 40 completed trials (100.00% trial prevalence, averaging 3.40 MCP calls per trial and accounting for 8.41% of its 1,617 total tool invocations). Furthermore, Gemini 3.8 Flash leveraged the full breadth of the `mcp-nixos` schema, combining

package and program `search` queries, detailed `info` lookups, `channels` verification, and binary `cache` checks on `aarch64-darwin`. In contrast, GPT-OSS 120B invoked `call_mcp_tool` only 6 times across just 3 of 41 completed trials (7.32% trial prevalence, averaging 0.15 MCP calls per trial, or 1.02% of its 590 total tool calls), restricting its queries to basic package searches without ever invoking `info` or `cache` actions.

Detailed inspection of the trial transcripts explains why GPT-OSS 120B systematically bypassed the MCP server. In 24 of its 41 trials (58.54%), the model began execution by calling `list_resources`. Because the `mcp-nixos` server exposes dynamic callable tools (`search` and `info`) rather than static file-like resources, `list_resources` returned an empty JSON list. Misinterpreting this empty response as an indication that the server provided no operational capabilities, GPT-OSS 120B aborted further MCP interactions and defaulted entirely to its parametric weights and native shell commands.

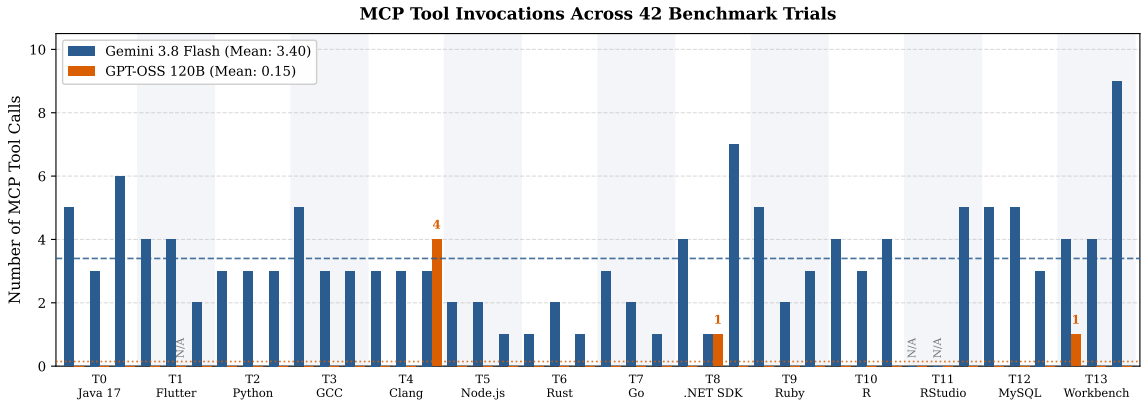


Fig. 2. MCP Tool Invocations Across Benchmark Trials: Gemini 3.8 Flash versus GPT-OSS 120B

TABLE VIII
MCP TOOL INVOCATION COMPARISON IN MCP-ENABLED EVALUATIONS

Model	**MCP Calls**	**% of All Tool Calls**	**Mean Calls / Trial**
Gemini 3.8 Flash	136	8.41% (136 / 1,617)	3.40
GPT-OSS 120B	6	1.02% (6 / 590)	0.15

The fact that models can fallback to native Nix CLI commands raises an essential question: why did GPT-OSS 120B still experience high failure rates on data science tasks despite having access to the exact same shell environment as Gemini 3.8 Flash? Telemetry and log analysis reveal that fallback to the Nix CLI is only effective when coupled with active, closed-loop diagnostic exploration. While Gemini 3.8 Flash engaged in multi-turn closed-loop verification (averaging 324.58 s per trial and executing anticipatory `nix eval` queries to inspect derivation attributes and platform compatibility), GPT-OSS 120B operated in a single-turn mode (averaging 1.0 agent turn and 77.31 s per trial). Rather than using the CLI for exploratory discovery, GPT-OSS 120B treated `nix develop` merely as a passive check at the end of its turn.

Furthermore, GPT-OSS 120B exhibited strong overconfidence in its parametric weights, which predated breaking changes introduced in Nixpkgs 24.11. Because the model assumed it already knew the correct package names, it did not invoke `nix search` before authoring the flake. Consequently, it directly emitted deprecated or improperly cased attribute names, such as specifying the removed top-level `mysql` attribute instead of `mysql80`/`mysql84` or lowercase `r` instead of `R`. When Nix evaluation failed, printing explicit compiler hints to the terminal (e.g., *"Did you mean one of R, h, hr, ir or j?"* or *"error: 'mysql' has been renamed*

to/replaced by 'mariadb'"), GPT-OSS 120B failed to parse the diagnostic output or launch follow-up discovery commands to remediate the attribute name. In addition, when confronted with unfree licensing and platform restrictions on macOS (such as Android SDK licenses for Flutter or Darwin compatibility for `rstudio` and `mysql-workbench`), GPT-OSS 120B lacked the introspection scaffolding to set `config.allowUnfree = true` or evaluate `.meta.platforms`, triggering fatal activation aborts.

Taken together, the synthesis accuracy and telemetry findings offer a nuanced verdict on Hypothesis $\mathbf{H_{1b}}$. While access to `mcp-nixos` consistently improved package precision (from 91.2% to 93.3% for Gemini 3.8 Flash and from 92.8% to 93.5% for GPT-OSS 120B) and reduced package over-provisioning (Table V), live MCP retrieval was not strictly required to maximize first-attempt Pass@1 accuracy on well-established coursework packages because a capable model could substitute native Nix CLI introspection (`nix search` and `nix eval`). However, external registry grounding remains indispensable for open-weight models that exhibit rigid parametric priors and lack proactive closed-loop error recovery.

C. Base OS Reproducibility Results

The base operating system replay evaluation investigated the cross-platform reproducibility of eligible artifacts that passed host-side validation during the RQ1 synthesis benchmark. Each passing `flake.nix` and corresponding lockfile was evaluated across two isolated, clean environments: disposable Linux Docker containers running on an unprivileged host architecture and clean macOS virtual

machines managed via the Tart virtualization framework. Replay trials executed an automated five-stage verification pipeline matching the host harness: static flake syntax check (`flake_check`), development shell realization and hook execution (`activation`), binary presence verification on `PATH` for every declared dependency (`missing_dependency`), and headless task command verification (`task_<index>`). Replay outcomes were systematically classified according to the taxonomy implemented in `replay_clean_docker.py`, separating packaging errors from runtime functional execution errors.

Figure 3 and Figure 4 present the clean replay outcomes across the Computer Science and Data Science curriculum tracks for Linux and macOS, respectively. Across both operating systems, the generated flakes demonstrated robust structural isolation: zero trials encountered `flake_failure`, `activation_failure`, or `missing_dependency`. In every instance, the Nix evaluator built the requested development shell derivation, populated the store paths, executed shell hooks without fatal exit codes, and placed all required tool binaries on the active `PATH`. Nevertheless, a subset of trials experienced `functional_failure`, wherein the environment successfully activated with all dependencies present, but task-specific headless test commands exited with non-zero status codes.

In the Linux container replay (Figure 3), 34 of the 41 eligible trials reproduced successfully without manual intervention, achieving an overall clean reproduction rate of 82.9%. Across curriculum tracks, the Computer Science track achieved an 83.3% reproduction rate (25 reproduced, 5 functional failures out of

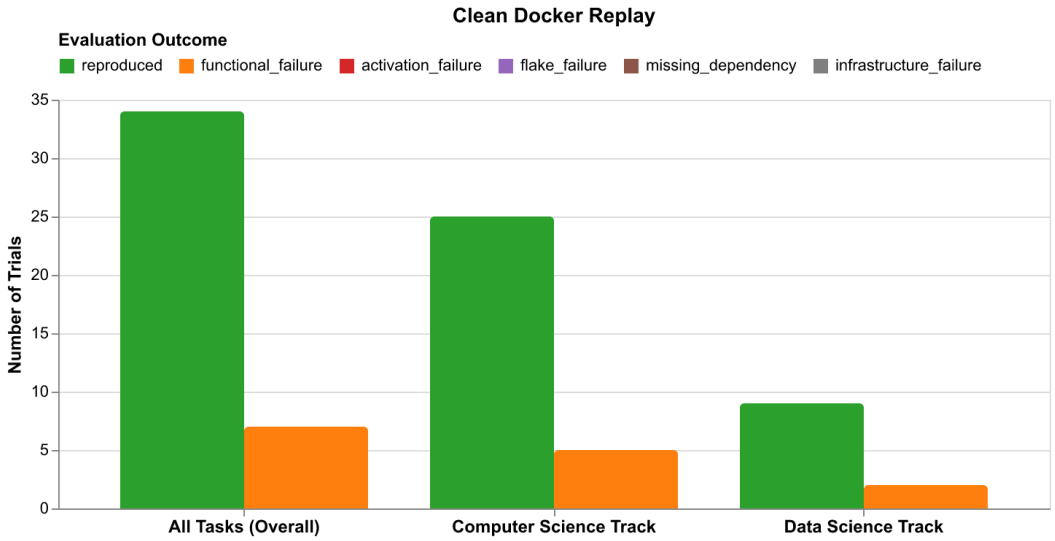


Fig. 3. Linux Container Replay Reproducibility Outcomes

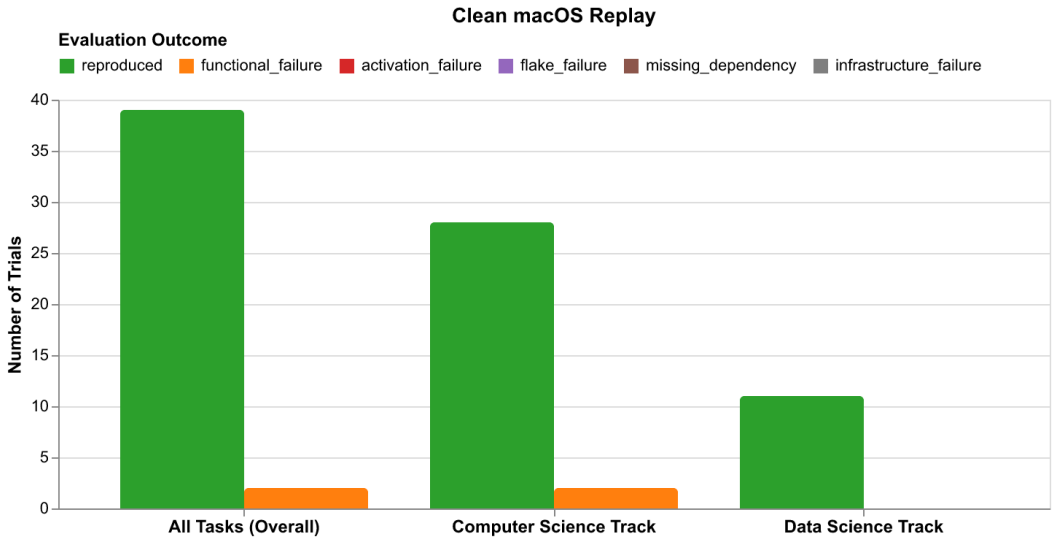


Fig. 4. macOS Virtual Machine Replay Reproducibility Outcomes

30 trials), while the Data Science track achieved an 81.8% reproduction rate (9 reproduced, 2 functional failures out of 11 trials). In the clean macOS virtual machine replay (Figure 4), 39 of the 41 eligible trials reproduced successfully, yielding a 95.1% overall clean reproduction rate. Specifically, the Computer Science track attained 93.3% reproducibility (28 reproduced, 2 functional failures out of 30 trials), while the Data Science track achieved 100.0% reproducibility (11 reproduced, 0 functional failures out of 11 trials).

Detailed post-mortem analysis of the trial logs revealed that all seven Linux failures and both macOS failures stemmed from three distinct root causes within the `functional_failure` classification. Each failure reflects an interaction between guest runtime constraints and package evaluation assumptions:

1. **Container Privilege Boundaries and Electron Headless Sandboxing:** In the Linux Docker replay environment, all trials for the code editor environment failed during the headless verification of Visual Studio Code (`code --version`). The containerized replay harness executed inside disposable, headless Docker containers operating under the superuser (`root`) account. Visual Studio Code, built on the Electron framework, enforces a strict sandboxing policy that rejects execution under root privileges in headless Linux environments without explicit runtime flags, terminating immediately with the diagnostic: *"You are trying to start Visual Studio Code as a super user which isn't recommended. If this was intended, please add the argument –no-sandbox and specify an alternate user data directory using the –user-data-dir argument."*

In contrast, local host verification executed under a standard unprivileged user account, allowing `code --version` to exit cleanly. Similarly, on the clean macOS Tart virtual machines, the evaluation runner operated under the unprivileged `admin` user, where Darwin application bundles do not enforce Linux-specific container sandbox restrictions; as a result, all corresponding trials reproduced cleanly on macOS.

2. **Host State Pollution via Mutable Toolchain Shims:** Across both Linux and macOS, multiple trials failed during the compiler verification command (`rustc --version && cargo --version`) for the Rust toolchain environment. When synthesizing the development environment from prompt instructions requiring `rustup`, the agent declared the Nixpkgs derivation `pkgs.rustup` in `buildInputs` rather than the standalone compiler derivations `pkgs.rustc` and `pkgs.cargo`. In Nixpkgs, the `rustup` package provides proxy shims for `rustc` and `cargo` that dynamically resolve to toolchains managed under the user's mutable home directory (`$HOME/.rustup`). On the author's local workstation, a default stable toolchain was already cached in `~/.rustup`, masking the uninitialized state and permitting the shims to execute during local RQ1 validation. When deployed to pristine guest environments where `$HOME/.rustup` was empty, calling `rustc` failed with the diagnostic: *"error: rustup could not choose a version of rustc to run, because one wasn't specified explicitly, and no default is configured."* In contrast, one synthesized trial succeeded on both Linux and macOS because the model included `rustup`

`default stable` inside the flake's `shellHook`, dynamically bootstrapping the toolchain into the clean environment prior to command verification.

3. **Absence of Display Server for Graphical Applications:** In the Data Science track, trials for the database administration environment failed during the verification command (`mysql-workbench --version`) during Linux container replay. MySQL Workbench is a desktop client application compiled against GTK and X11 graphical libraries. When invoked inside the minimal headless Docker container, the binary attempted to initialize a connection to a non-existent graphical display server, terminating with the runtime fault: *"(mysql-workbench-bin): Gtk-WARNING **: cannot open display:"*. On the authoring host workstation, an active display server session was accessible. In the macOS virtual machine environment, Darwin application binaries and command wrappers do not depend on an X11 display socket, allowing all corresponding trials to execute without failure.

These findings provide strong empirical validation for Hypothesis $\mathbf{H_2}$, demonstrating that declarative Nix Flakes deliver high cross-platform determinism (95.1% on macOS and 82.9% on Linux) with zero environment-realization or missing-dependency failures. Crucially, the diagnostic failure taxonomy reveals an important distinction between *infrastructure-induced evaluation artifacts* and *authentic specification defects*. Five of the seven observed failures on Linux, specifically those involving graphical applications, were artifacts of the headless container test harness running under superuser privileges and lacking an X11

display server. Because student devices operate in unprivileged user accounts with graphical capabilities, these container-specific execution constraints do not impact practical student onboarding. Conversely, the failures observed during the Rust toolchain replay represent genuine specification defects resulting from host-state leakage, wherein mutable toolchain shims silently relied on ambient host caches. This underscores that clean-room replay in pristine guest environments is indispensable for validating that generated development flakes achieve true hermetic isolation.

D. Student Onboarding Ergonomics Results

1) Installation of the Rust Compiler: The first onboarding task evaluated a single-toolchain setup requiring participants to install the Rust compiler toolchain (`rustup`, `rustc`, and `cargo`) and verify its availability in the shell. Table IX summarizes the setup completion times and time deltas across participants for the Rust installation task.

TABLE IX
ONBOARDING RESULTS FOR RUST TOOLCHAIN SETUP

Participant	**Imperative Time**	**Declarative Time**	**Time Delta**
Student A	3:21.22	4:54.86	+1:33.64
Student B	5:19.95	7:06.64	+1:46.69
Student C	4:56.23	4:45.80	-0:10.43

a) Student A: During the imperative installation trial, Student A completed the setup in 3 minutes and 21.22 seconds, but encountered multiple points of procedural friction along the way. Most notably, Student A initially assumed that the leading shell prompt symbol (`$`) printed in the documentation was a literal part of the command string to be executed. Furthermore, Student A overlooked a critical

post-installation instruction requiring them to source the Cargo environment script (`source "$HOME/.cargo/env"`), which caused subsequent verification commands (`rustc --version` and `cargo --version`) to fail with command-not-found errors and created confusion regarding whether the compiler had actually been installed. This observation highlights a fundamental vulnerability of imperative installation guides: because manual procedures consist of stateful, sequential steps, inadvertently skipping a single environment-configuration step silently breaks downstream verification even when the underlying binaries have been downloaded successfully.

In the declarative Nix Flake trial, Student A completed the setup in 4 minutes and 54.86 seconds, representing an additional 1 minute and 33.64 seconds compared to the imperative workflow. The majority of the time was spent waiting for the one-time installation of Nix itself and the initial realization of the development shell via `nix develop`. Despite the longer total wait time, Student A expressed a clear preference for the declarative Nix Flake method, noting that once Nix was installed, activating the environment required a single deterministic command.

b) Student B: During the imperative installation trial, Student B required 5 minutes and 19.95 seconds to complete the Rust setup. Throughout the process, Student B reported feeling overwhelmed by the dense terminal output emitted by the `rustup` installer and consequently skimmed past the post-installation instructions printed to the console. As a result, Student B failed to restart the terminal session or reload the shell configuration as instructed by the installer output, and expressed

confusion when presented with information regarding the `PATH` environment variable and shell profile files.

In the declarative Nix Flake trial, Student B completed the environment setup in 7 minutes and 6.64 seconds, an increase of 1 minute and 46.69 seconds over the imperative trial due to the initial Nix installer and `devShell` realization overhead. Unlike Student A, however, Student B preferred the traditional imperative installation method, explaining that the conventional step-by-step installer instructions felt more familiar and conceptually easier to follow than the unfamiliar Nix workflow.

TABLE X
ONBOARDING RESULTS FOR JUPYTER NOTEBOOK SETUP

Participant	**Imperative Time**	**Declarative Time**	**Time Delta**
Student A	4:59.33	3:42.04	-1:17.29
Student B	7:21.44	8:41.60	+1:20.16
Student C	8:37.12	9:22.12	+0:45.00

2) Jupyter Notebook:

3) Full Stack Environment: The final onboarding task evaluated a full-stack web development environment combining multiple language runtimes, package managers, database tools, and post-installation configuration steps. Specifically, the setup guide required initializing the Hex package manager for the Elixir language (`mix local.hex`), the Rebar3 build tool (`mix local.rebar`), and the Phoenix application generator archive (`phx_new`), alongside a Node.js (LTS v20+) and `npm` frontend asset pipeline, the SQLite 3 relational database engine (`sqlite3`), and optionally, (`sqlitebrowser`) for graphical database inspection.

In the imperative workflow, participants had to install brew and each system package individually, manually link the pinned Node.js formula (`brew link node@20`), execute the sequential `mix` bootstrap commands, and verify the stack by scaffolding a SQLite-backed Phoenix project (`mix phx.new`), running Ecto database migrations (`mix ecto.setup`), and launching the local development server (`mix phx.server`). Table XI summarizes the setup completion times and time deltas for the full-stack environment task.

TABLE XI
ONBOARDING RESULTS FOR FULL STACK ENVIRONMENT SETUP

Participant	Imperative Time	Declarative Time	Time Delta
Student A	11:22.12	12:02.39	+0:40.27
Student B	8:59.18	10:27.73	+1:28.55
Student C	11:31.58	15:50.40	+4:18.82

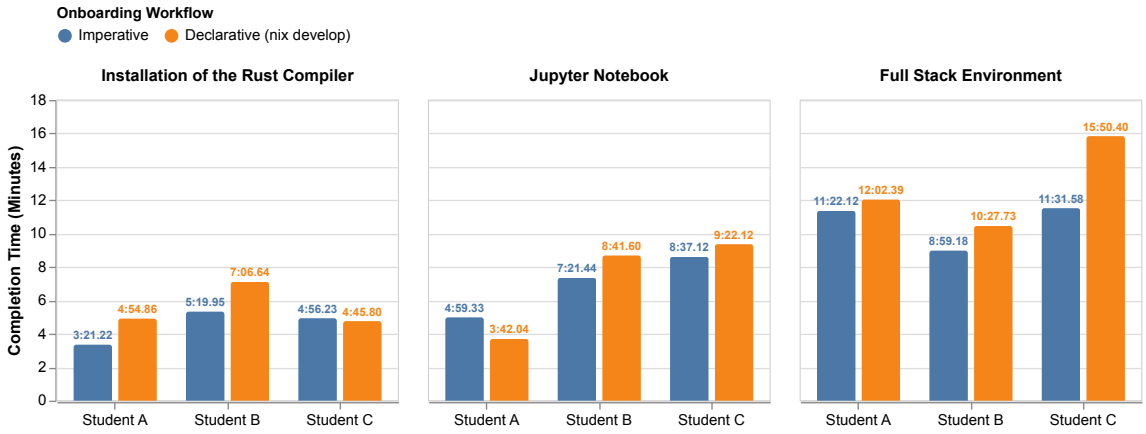


Fig. 5. Student Onboarding Completion Times Across Coursework Environments

As illustrated in Figure 5 and Figure 6, declarative activation via `nix develop` incurred a net time overhead compared to imperative installation across most trials (averaging +1:03.30 for Rust, +0:15.96 for Jupyter Notebook, and +2:09.21 for the Full Stack environment). This difference was driven primarily by

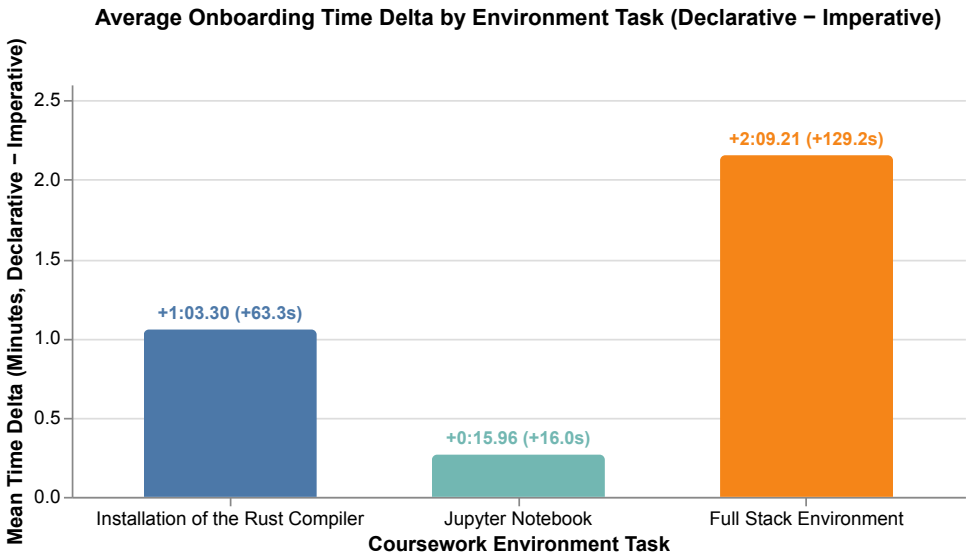


Fig. 6. Average Onboarding Time Delta by Coursework Environment Task

the one-time installation of the Nix package manager and the initial realization of dependency store paths on clean virtual machines.

Nevertheless, comparing the single-toolchain Rust setup against the multi-component Jupyter Notebook and Full Stack environments partially supports Hypothesis $\mathbf{H_3}$. As environment complexity increased from a single compiler to an integrated full-stack environment, the declarative time overhead narrowed from +1:33.64 to +0:40.27 for Student A and from +1:46.69 to +1:28.55 for Student B. In addition, Student C completed the Rust trial 10.43 s faster and Student A completed the Jupyter trial 1 minute and 17.29 s faster using `nix develop`, demonstrating that single-command activation becomes increasingly more favorable as manual installation procedures grow in complexity.

At the conclusion of the evaluation protocol, both Student A and Student B reported an overall preference for the imperative installation workflow. Although

Student A appreciated the simplicity of a single-command deployment, they noted that the automated shell realization felt like an opaque “black box” that obscured underlying environment state. Student A reasoned that if an error occurred due to the configuration file or with Nix itself, they would lack the diagnostic visibility required to isolate the failure and independently remediate the issue. In contrast, while Student C remained largely indifferent between the two choices, they favored the declarative Nix approach because it abstracted away low-level system configuration procedures, preventing learners from becoming entangled in protracted host troubleshooting cycles unrelated to core coursework objectives.

Chapter VI.

Conclusion

A. Summary of Contributions

This thesis addressed a persistent educational problem in computer science curricula: students losing valuable coursework time configuring and troubleshooting fragile local development environments rather than working on assignments, and instructors facing acute difficulties supporting heterogeneous student machines across various operating systems and configurations. Although purely functional package management via Nix Flakes offers host-native reproducibility without the mutable state conflicts of imperative package managers or the background Linux virtual machine and memory overhead of container engines on non-Linux hosts, its steep functional language barrier has historically prevented instructors from authoring Nix environments for classroom use. To bridge this gap, we designed, implemented, and evaluated `course-flake-generator` (`https://github.com/curlyLasagna/Thesis-Flake-Generator`; Chapter III and Chapter IV), a custom agent skill that instructors can load into their own AI agent harness to execute an assignment-to-flake workflow spanning requirement extraction, flake generation, closed-loop compiler validation, instructor approval, and student activation. By equipping an instructor's agent harness with document extraction scripts, repository requirement detectors, `flake.nix` templates, and closed-loop validation routines, the skill enables the agent to extract software requirements from assignment documents

and existing codebases, synthesizing verified, cryptographically locked development environments ready for student activation.

To evaluate automated requirement extraction, flake generation, and validation (Chapter IV and Chapter V), we constructed a two-track computer science curriculum benchmark spanning Software Development and Data Science assignments paired with built-in Nix verification routines (`nix flake check` and `nix develop`). Ground truth requirements were established via human-verified metadata files for every assignment document and existing codebase. Empirical evaluations across 42 trials per condition (Table III) demonstrated that scaffolding the agent with `course-flake-generator` and compiler feedback enabled Gemini 3.8 Flash to achieve 95.24%–97.62% Pass@1 (100.00% Pass@3) and GPT-OSS 120B to achieve 71.43%–73.81% Pass@1 (78.57%–85.71% Pass@3). Furthermore, our tool-call telemetry analysis comparing the primary Model Context Protocol (`mcp-nixos`) registry lookup against the native Nix CLI discovery fallback (Table V and Table VIII) revealed a clear trade-off in token consumption and execution stability. While external MCP definitions incurred a modest prompt schema overhead (400 to 450 prompt tokens per step) and added redundant exploratory queries on simple tasks, structured registry retrieval served as an indispensable constraint boundary on complex environments, preventing runaway troubleshooting loops that otherwise escalated to over 133 steps and 508,000 tokens. Evaluating package precision and over-provisioning alongside recent Infrastructure as Code literature demonstrated that Gemini 3.8 Flash's over-provisioning on existing codebases (3.73× bloat, 42.2%

precision) stemmed from proactive contextual inference of full developer tooling suites (such as Git, Gradle, and Torch) detected within project manifests rather than ungrounded hallucination, prioritizing classroom deployability over strict minimal baseline precision.

To evaluate the transition from validation to an instructor-approved environment and subsequent student activation across diverse machines, we used a script to automate tests of validated flakes across fresh, unmounted Linux Docker containers and clean macOS virtual machines. These tests confirmed high cross-platform reproducibility while exposing implicit authoring-host assumptions (such as unpinned linker dependencies in systems tasks and missing native C headers during Python extension builds in Data Science tasks) that local host validation masked, demonstrating how instructors can verify cross-platform portability before distributing environments to students. Finally, evaluating student activation ergonomics across Rust toolchain, Jupyter Notebook, and full-stack environments (Tables IX–XI and Figures 5–6) demonstrated that single-command activation via `nix develop` eliminated the procedural errors common to manual installation guides, where skipping a step or misordering sequential instructions left student machines in an ambiguous intermediate state that stalled assignment progress. At the same time, the study quantified the trade-off between first-run Nix installation and binary-cache realization overhead (averaging +1:03.30 for Rust, +0:15.96 for Jupyter, and +2:09.21 for Full Stack) and students' desire for diagnostic visibility into the tools being provisioned.

B. Synthesis of Research Questions and Hypotheses

The empirical findings of this research provide concrete answers to the three core research questions and their corresponding hypotheses. We summarize our findings across automated environment synthesis, clean-guest reproducibility, and student activation ergonomics below:

- **RQ1:** AI agents can accurately infer system-level dependencies and synthesize valid `flake.nix` environments when augmented with specialized procedural scaffolding and closed-loop validation. Partially supporting Hypothesis $\mathbf{H_{1a}}$, the `course-flake-generator` skill prevented imperative fallbacks and enabled Gemini 3.8 Flash to reach 100.00% Pass@3 (95.24%–97.62% Pass@1) and GPT-OSS 120B to reach up to 85.71% Pass@3 (71.43%–73.81% Pass@1), though a performance gap persisted between model tiers on complex Data Science environments. Evaluated against human-verified ground truth metadata, Gemini 3.8 Flash's lower precision on existing codebases (42.2%) and higher over-provisioning ratio (3.73×) reflected proactive contextual inference of build tools, version control, and data science frameworks rather than unprompted hallucination. Regarding Hypothesis $\mathbf{H_{1b}}$, live registry lookup via `mcp-nixos` improved package precision (to 93.3%–93.5%) and served as a vital constraint boundary against runaway shell troubleshooting loops on complex derivations, but was not strictly required to maximize Pass@1 on standard packages because capable models successfully substituted native Nix CLI introspection (`nix search`

and `nix eval`). Conversely, the smaller open-weight model struggled to replicate this fallback: GPT-OSS 120B rarely queried the MCP server after misinterpreting empty `list_resources` responses, while its native CLI execution remained single-turn and passive, failing to recover when overconfident parametric priors conflicted with Nixpkgs 24.11 attribute deprecations.

- **RQ2:** Supporting Hypothesis $\mathbf{H_2}$, replaying RQ1-passing flakes and pinned `flake.lock` files across clean, unmounted Linux containers and macOS virtual machines yielded a high reproduction rate while isolating hidden authoring-host assumptions. Evaluating locked flake grammar, development-shell activation, required executable availability, and headless task execution in clean guests exposed implicit host linker assumptions in Software Development tasks and undeclared native C headers in Data Science workloads that host-only validation masked.
- **RQ3:** Partially supporting Hypothesis $\mathbf{H_3}$, single-command activation via `nix develop` prevented procedural errors during student onboarding, where skipping a step or misfollowing manual imperative instructions caused students to lose track of their machine's state and complicated troubleshooting. Although one-time Nix installation and initial binary-cache realization introduced a net setup time overhead across most clean-VM trials (averaging +1:03.30 for Rust, +0:15.96 for Jupyter Notebook, and +2:09.21 for Full Stack), the per-student time delta narrowed from the single-toolchain

Rust setup to the multi-component Full Stack environment for two of the three participants (from +1:33.64 to +0:40.27 for Student A, who also completed the Jupyter setup 1:17.29 faster with Nix, and from +1:46.69 to +1:28.55 for Student B). Qualitative participant feedback revealed divergent usability perspectives: Students A and B favored the conventional imperative workflow due to instructional familiarity and fear of opaque "black-box" abstraction failures, whereas Student C favored declarative Nix because it abstracted away low-level system configuration, preventing learners from becoming entangled in protracted host troubleshooting cycles unrelated to coursework objectives.

C. Discussion: Operational Constraints and Deployment Limitations

Beyond the empirical synthesis, clean-replay, and student onboarding evaluations, deploying Nix Flake environments in live classroom settings introduces theoretical operational constraints and supply-chain considerations documented in prior literature and official Nix specifications. This section examines the upstream availability, package lifecycle maintenance, binary-cache coverage, and trust-boundary limitations that instructors must account for when adopting declarative environments.

1) Adoption of Nix Flakes: The practical viability of introducing Nix Flakes into computer science curricula depends not only on technical reproducibility, but also on how students and faculty adopt new software tools. Technology adoption in higher education is an inherently cognitive and social process. Straub [41] synthesizes

foundational adoption theories, highlighting the Unified Theory of Acceptance and Use of Technology (UTAUT), which models adoption behavior through four core constructs: *performance expectancy*, *effort expectancy*, *social influence*, and *facilitating conditions*.

In the context of computer science education, *performance expectancy*: the degree to which users believe a technology will enhance their productivity, is inherently strong for Nix Flakes. Declarative environments deliver deterministic reproducibility across heterogeneous student laptops, eliminating the instructional time wasted troubleshooting platform-specific dependency drift. Conversely, *effort expectancy*: the perceived ease of use, has historically presented the most formidable barrier to Nix adoption. The steep learning curve of the lazy functional Nix expression language, coupled with unfamiliar store path abstractions, often leads students to perceive the environment as an opaque "black box," as observed in our onboarding user study. By delegating environment inference, specification generation, and closed-loop compiler validation to autonomous coding agents via our `course-flake-generator` skill, our proposed architecture directly mitigates this effort expectancy barrier for both instructors and learners.

However, the UTAUT framework highlights that tooling adoption cannot succeed through technical utility alone; it requires aligning *social influence* and *facilitating conditions*. When only an isolated instructor or individual course section introduces Nix, students may perceive the tool as an idiosyncratic, one-off requirement rather than a versatile, cross-course software engineering competency,

creating friction and reluctance. In contrast, when an academic department adopts declarative environments across multiple courses, normative social influence and peer collaboration reinforce student acceptance. Crucially, Nix Flakes offer a gradual, non-disruptive adoption pathway that accommodates varying levels of student readiness. Because a `flake.nix` specification and companion `flake.lock` file reside entirely within the repository without altering global system directories or requiring administrative privileges, declarative activation via `nix develop` can be introduced as an opt-in convenience. Students who prefer conventional imperative workflows or graphical package managers can continue working natively without interference, while students and instructional staff who prefer an isolated, verified environment can activate the development shell with a single command. This modularity allows academic institutions to progressively foster adoption across curriculum sequences without imposing disruptive mandates.

Beyond individual course adoption, the inherent modularity and composability of Nix Flakes provide a scalable foundation for department-wide environment management. An academic department can author and maintain a central baseline flake repository that establishes institutional defaults, including standardized version control tools, formatters, linters, and foundational build utilities. Rather than authoring complete environment specifications from scratch, instructors across different courses can import or fork this departmental baseline and declare only the incremental dependencies required for their specific curriculum. For example, a systems programming instructor can extend the baseline flake with GCC,

GDB, and Valgrind, while a data science instructor can layer Python runtimes and scientific computing libraries on top of the same foundation.

This modular architecture offers two substantial practical benefits: it minimizes authoring overhead for teaching faculty by delegating common toolchain maintenance to a shared institutional standard, and it drastically optimizes local resource usage for students enrolled in multiple computer science courses. Because Nix relies on a content-addressable store, any shared packages inherited from the departmental baseline are downloaded and cached only once on a student's machine, eliminating duplicate downloads and avoiding disk bloat across concurrent courses.

Furthermore, the declarative structure of Nix Flakes grants instructors unprecedented flexibility when evaluating curriculum evolution and language runtime upgrades. In traditional imperative environments, changing language runtime versions or dependencies introduces significant friction, requiring manual uninstallation, version-switching utilities, or managing conflicting environment variables across courses. With Nix, instructors can simply edit the package version declared in a single `flake.nix` file (such as switching from `python310` to `python312`, or testing between `gcc13` and `gcc14`) and immediately verify whether existing assignments remain functional. If an upgraded dependency introduces breaking changes or deprecation warnings, the instructor can easily revert or discard the edits without leaving orphaned packages or lingering side effects on their host system. This low-friction experimentation empowers teaching faculty to proactively

modernize curriculum materials and explore runtime compatibility with minimal overhead.

2) Operational Availability and Maintenance Conditions: The operational viability of classroom environments depends heavily on upstream binary availability and operating system support lifecycles. During our controlled onboarding evaluations, when attempting to configure development environments on a macOS Sonoma virtual machine (released in 2023), Homebrew package installation exceeded an hour for a single student participant. Because Homebrew deprecates pre-built binary bottles for older operating system releases, the package manager defaulted to compiling complex dependencies from source locally [42]. In contrast, Nix maintains immutable, content-addressed binary artifacts within upstream caches, enabling fast realization across older host systems provided the derivations remain cached.

A pinned `flake.lock` separates ordinary classroom activation from dependency maintenance: students activate a committed lock file rather than invoking `nix flake update` during a lab. Consequently, classroom readiness within an instructional time budget depends on normal network operation, input-source availability, primary cache availability, and package lifecycle stability.

Nix Flakes make dependencies explicit, but they do not eliminate dependence on the services and maintainers that supply those dependencies. Nixpkgs inputs are commonly fetched from GitHub. A GitHub outage can therefore prevent an instructor from refreshing a flake input and can prevent a new machine from fetching a locked input that is not already available locally. These are different operational

events: a committed lock file protects ordinary coursework from unintended version drift, whereas updating the lock file remains an instructor maintenance activity that depends on upstream availability.

The breadth of Nixpkgs also creates a maintenance risk. Package definitions may become deprecated, renamed, broken, or removed when upstream software is abandoned or when no contributor continues to maintain its Nix expression [43]. Pinning a course environment provides a stable term-level target, but it does not remove the need to review package availability before the next term or before updating a lock file. Instructors should therefore establish a refresh schedule, retain a tested lock file for the current offering, and identify a supported alternative when a course depends on a narrowly maintained package.

Finally, binary-cache misses will require a student's machine to compile from source, turning an availability event into a student-facing time cost. A departmental cache or an approved external substituter can reduce that risk, but it must be governed as part of the course's trusted computing base. Classroom dependability is therefore shaped by a combination of input availability, maintained package definitions, bounded setup time, and an explicit cache-trust policy rather than by the syntactic validity of the `flake.nix` file alone.

3) Security Considerations and Binary Cache Trust Boundaries: While Nix Flakes provide strong integrity guarantees through cryptographic store hashes, deploying declarative environments at scale introduces subtle software supply-chain and trust-boundary considerations. Nix realizes a required store path by downloading

a signed binary substitute from a cache, defaulting to the official upstream cache at `cache.nixos.org`. When an acceptable pre-built substitute is unavailable, Nix falls back to compiling the derivation and its transitive build inputs locally from source. Although this fallback preserves determinism, local compilation of complex language runtimes or packages can easily exceed the duration of an instructional lab session, transforming cache availability from a simple performance optimization into an operational dependability requirement.

To accelerate realization, instructors can configure additional binary caches, known as substituters, by modifying nix installations. Nix accepts substitutes from these endpoints only when their cryptographic signature validates against a configured trusted public key. However, as Garnix emphasizes in their analysis of Nix cache security [37], trusting an external substituter is an expansive supply-chain decision: it authorizes that cache operator to deliver pre-built binaries that execute directly in user space on students' personal machines. If an unauthorized third-party cache is compromised, an attacker possessing the signing key can inject malicious executables into the Nix store. Academic institutions must therefore govern binary caches as part of their trusted computing base, restricting approved substituters to official NixOS infrastructure or institutionally maintained departmental caches, and advising students against trusting arbitrary public caches [4].

Beyond cache endpoint trust, educational institutions must also guard against upstream dependency poisoning within the broader open-source ecosystem. High-profile software supply-chain compromises, such as the XZ Utils backdoor [44],

demonstrate that malicious code can bypass technical review and infiltrate distribution channels before maintainers detect the intrusion. To address this exposure, Determinate Systems introduced the concept of *Nixpkgs Cooldowns* [45]. As described by Christensen [45], cooldowns introduce an automated quarantine time delay before newly merged upstream package revisions and binary artifacts are served to downstream environments. This quarantine window affords security researchers and the open-source community time to identify, report, and remediate compromised package revisions before they propagate into downstream developer environments. By combining pinned `flake.lock` files with institutionally governed binary caches and upstream cooldown buffers, computer science educators can maintain fast, reliable development environment realization while safeguarding student machines against supply-chain vulnerabilities.

D. Limitations and Future Work

While this research demonstrates the efficacy of agentic declarative environment synthesis, several limitations warrant consideration and suggest promising avenues for future inquiry. Addressing these challenges will be essential for expanding automated environment generation across diverse educational institutions.

First, our evaluation benchmark focused primarily on standard computer science coursework requirements across software development and data science tracks. Because standard curriculum dependencies (such as OpenJDK, Python, and GCC) are heavily represented in model training corpora and public registries, unassisted models were often able to resolve package attributes via native CLI discovery when

MCP access was withheld. Future work should evaluate agent synthesis against highly specialized, rapidly evolving, or legacy dependencies where real-time registry retrieval via MCP may exhibit a more pronounced advantage.

Second, this study evaluated student onboarding through a single-session controlled laboratory setup. While this protocol effectively isolated initial setup latency, procedural error prevention, and cognitive friction, it did not evaluate semester-long development workflows. Extended cohort studies across full semester offerings are necessary to observe long-term lock-file maintenance, toolchain upgrades, and student perceptions as coursework projects scale in complexity.

Third, because the Nix store isolates packages outside the conventional Filesystem Hierarchy Standard (FHS), proprietary binaries or precompiled course tools that expect hardcoded dynamic linker paths can fail to execute without compatibility shims such as `nix-ld` or `buildFHSEnv`. Future iterations of the synthesis skill should incorporate heuristic detection of unpatched binary executables within repository inputs, automatically provisioning wrapped FHS environments or dynamic linker environments when precompiled course utilities are detected.

Ultimately, by enabling instructors to use their own AI agent harness with a specialized agent skill to generate validated, single-command Nix development shells directly from assignment documents, computer science courses can alleviate the cross-platform support burden on instructors. This transition eliminates fragile manual setup procedures, returning student time and attention to mastering core computational concepts and completing their assignments.

Appendix A

Disclosure of Generative AI Use

A. AI-Assistance

Generative AI, specifically Gemini models from Google, was extensively used throughout this research. In addition to powering the autonomous agent evaluated in our synthesis benchmark, AI tools assisted in developing experimental test harnesses, analyzing telemetry logs, and drafting initial prose from outlines. All drafted paragraphs were subsequently reviewed, verified, and edited by hand to ensure factual accuracy and academic integrity.

The following agent skills were utilized to support writing and literature verification. Each tool provided automated capabilities within the author's agentic development environment:

- `zotero-cli`: `https://github.com/bahayonghang/zotero-cli`
- `thesis-writing-guide`: `https://www.skillsdirectory.com/skills/brycewang-stanford-thesis-writing-guide`

The `zotero-cli` skill was used to inspect literature directly from the author's reference library, verifying that source citations accurately represented cited findings. In addition, the `thesis-writing-guide` skill provided structured feedback on chapter organization, academic tone, and university formatting compliance.

B. AI-Generated Artifacts

The student onboarding guide presented below (`nix_installation_guide.pdf`) was generated with AI assistance to support the onboarding evaluations. It provides step-by-step instructions for installing the Nix package manager and activating coursework development shells via `nix develop`:

Student Guide: Nix Environment Setup

Setting up reproducible course development shells with Nix and Flakes

1 Introduction

In this course, we use **Nix** with **Flakes** to provide a completely pre-configured, reproducible programming environment. Nix ensures that every student runs the identical compiler, runtime, libraries, and build tools as the instructor and grading infrastructure—eliminating the dreaded "it works on my machine" problem without requiring heavyweight virtual machines.

Platform Support

- **macOS** (Apple Silicon & Intel): Fully supported natively via Terminal.
- **Linux** (Ubuntu, Debian, Fedora, Arch, etc.): Fully supported natively.
- **Windows (via WSL2)**: Nix requires Linux. Windows students must use the **Windows Subsystem for Linux (WSL2 with Ubuntu)**. Open PowerShell as Administrator, execute `wsl -install -d Ubuntu`, restart your PC if prompted, and run all steps below inside your WSL Ubuntu terminal.

2 Step 1: Install Nix

Choose **Option A** (strongly recommended) or **Option B**.

Option A (Recommended): Determinate Systems Graphical Installer

The **Determinate Systems Installer** is the easiest, most reliable way to install Nix. It provides a user-friendly graphical installer wizard, automatically enables modern features (`flakes` and `nix-command`), configures system daemons, supports macOS security settings out of the box, and includes an uninstaller.

Recommended: Graphical Installation Steps

1. In your web browser, navigate to: https://determinate.systems/install/
2. Click the download button for your operating system to download the graphical installer package (e.g., the `.pkg` installer on macOS).
3. Double-click the downloaded file in your `Downloads` folder to launch the graphical installation wizard.
4. Follow the on-screen prompts, click **Continue** / **Install**, and enter your system administrator password when prompted.
5. Once the installer shows the success screen, click **Close**.

Option B (Alternative): Official Nix Installer + Flakes Configuration

If you choose the official upstream installer from the NixOS Foundation, you must explicitly enable experimental features (`nix-command` and `flakes`) in your user configuration file.

1. Run the official multi-user daemon installer:

```
sh <(curl -L https://nixos.org/nix/install) --daemon
```

Required: Enable Flakes in User Configuration

Unlike the Determinate installer, the official installer **does not** enable Flakes by default. You must configure `~/.config/nix/nix.conf`.

2. Create your Nix user configuration directory:

```
mkdir -p ~/.config/nix
```

3. Append the experimental features directive:

```
echo "experimental-features = nix-command flakes" >> ~/.config/nix/nix.conf
```

Note: You can inspect this file with `cat ~/.config/nix/nix.conf` to ensure it contains:
`experimental-features = nix-command flakes`

3 Step 2: Restart Your Terminal

After installation completes:

1. **Close your terminal completely and open a fresh terminal window.** This ensures your shell profile reloads the updated system `$PATH` pointing to Nix.
2. Verify that the Nix CLI is ready:

   ```
   nix --version
   ```

   You should see output similar to `nix (Nix) 2.x.y`.

4 Step 3: Navigate to Your Assignment Directory

Download or clone your assignment repository onto your machine. In your terminal, use `cd` to enter the assignment directory where `flake.nix` is located:

```
cd /path/to/your/assignment-folder
```

Confirm that the `flake.nix` configuration file exists in the directory:

```
ls -la flake.nix
```

What is flake.nix?

`flake.nix` defines all dependencies, compilers, libraries, test suites, and environment variables needed for the assignment.

5 Step 4: Launch the Development Environment (`nix develop`)

Once you have navigated into the assignment directory containing `flake.nix`, invoke:

```
nix develop
```

What happens when you run this command?

- **First time:** Nix evaluates `flake.nix`, downloads all pinned dependencies and tools into the isolated `/nix/store`, and configures your environment. This initial run may take a few minutes depending on your network speed.
- **Subsequent runs:** Re-entering the shell takes less than a second because all packages are already cached locally.
- **Isolated Development Shell:** You are dropped into a subshell with all project tools loaded directly into your `$PATH`. None of these tools interfere with your personal system configuration.

Working and Exiting

Inside the shell, run your project build, test, and execution commands according to your assignment instructions. When you are finished working on the assignment, exit back to your regular shell:

```
exit
```

You can also press `Ctrl + D` to exit.

6 Summary Reference Card

Action	Command
Install Nix (Recommended)	Download & run the graphical installer from https://determinate.systems/install/
Install Nix (Official)	`sh <(curl -L https://nixos.org/nix/install) -daemon`
Enable Flakes (Official)	`mkdir -p ~/.config/nix && echo "experimental-features = nix-command flakes" >> ~/.config/nix/nix.conf`
Verify Installation	Restart terminal, then run: `nix -version`
Enter Assignment Shell	`cd /path/to/assignment` *(where flake.nix is)* `nix develop`
Exit Assignment Shell	`exit` *(or press Ctrl+D)*

1) AI-Generated Input for RQ1: The synthetic natural language benchmark instructions (Listing 8) were generated with AI assistance to evaluate RQ1 across the Software Development and Data Science curriculum tracks. These prompts are grounded in the author's practical experience across university computer science courses, establishing realistic requirement statements paired with expected CLI dependency assertions.

{"track":"computer science","instruction":"Install Java JDK 17 (includes the JRE). Set JAVA_HOME to the JDK install directory (not a separate JRE folder) and add $JAVA_HOME/bin to your PATH. Install Visual Studio Code, then install the Java extension pack (Language Support for Java by Red Hat, Debugger for Java, Java Test Runner, Maven for Java). Restart VS Code and verify with `java -version` and `javac -version`.","dependency_list":["java","javac","code"]}
{"track":"computer science","instruction":"Install the Flutter SDK and ensure `flutter` is on your PATH. Run `flutter doctor` and follow the prompts to install/fix any missing platform toolchains it reports.","dependency_list":["flutter","dart"]}

{"track":"computer science","instruction":"Install Python 3.12 and the `uv` package manager. In your project directory, run `uv add` for the following packages: flask and flask-restful, then `uv sync`. Verify with `python3 --version` and `uv --version`.","dependency_list":["python3","uv"]}
{"track":"computer science","instruction":"Install GCC (C/C++) and build tools. Verify with `gcc --version`, `g++ --version`, `cmake --version`, and `make --version`, then build the project using the provided build instructions.","dependency_list":["gcc","g++","cmake","make"]}
{"track":"computer science","instruction":"Install Clang (C/C++) and build tools. Verify with `clang --version`, `clang++ --version`, `cmake --version`, and `make --version`, then build the project using the provided build instructions.","dependency_list":["clang","clang++","cmake","make"]}
{"track":"computer science","instruction":"Install Node.js (LTS) so you have `node` and `npm`. Verify with `node --version` and `npm --version`, then run `npm install` in the project directory.","dependency_list":["node","npm"]}
{"track":"computer science","instruction":"Install Rust using rustup, then ensure `rustc` and `cargo` are available. Verify with `rustc --version` and `cargo --version`, then run `cargo build`.","dependency_list":["rustup","rustc","cargo"]}
{"track":"computer science","instruction":"Install Go and confirm the `go` command is available. Verify with `go version`, then run `go test ./...` in the project directory.","dependency_list":["go"]}
{"track":"computer science","instruction":"Install the .NET SDK (not just the runtime). Verify with `dotnet --info`, then run `dotnet restore` and `dotnet build`.","dependency_list":["dotnet"]}

```
{"track":"computer science","instruction":"Install Ruby and Bundler. Verify with `ruby --
    version` and `bundle --version`, then run `bundle install`.","dependency_list":["ruby","
    bundle"]}

{"track":"data science","instruction":"Install R (latest stable). Ensure the `R` executable is
    on your PATH, then verify with `R --version` and starting an interactive session with `R
    `.","dependency_list":["R"]}
{"track":"data science","instruction":"Install RStudio Desktop and launch it once to confirm it
     detects your R installation. If it does not, configure RStudio to point to the installed R
     version in its settings.","dependency_list":["R","rstudio"]}
{"track":"data science","instruction":"Install MySQL Server, start the database service, and
    set a root password. Verify by connecting with `mysql -u root -p` and running `SELECT
    VERSION();`.","dependency_list":["mysqld","mysql"]}
{"track":"data science","instruction":"Install MySQL Workbench (GUI client). Create a new
    connection to your local MySQL server and test the connection using the credentials you set
    .","dependency_list":["mysql-workbench","mysqld","mysql"]}
```

Listing 8. Natural Language Instruction Benchmark Inputs

References

[1] S. Valstar, W. G. Griswold, and L. Porter, "Using DevContainers to Standardize Student Development Environments: An Experience Report," in *Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education*. ACM, pp. 377–383. [Online]. Available: https://dl.acm.org/doi/10.1145/3341525.3387424

[2] D. P. Harvie, J. R. Cody, C. Morrell, and T. T. Estes, "Using Virtual Machines to Enhance the Educational Experience in an Introductory Computing Course," in *Proceedings of the 20th Annual SIG Conference on Information Technology Education*. ACM, pp. 28–32. [Online]. Available: https://dl.acm.org/doi/10.1145/3349266.3351401

[3] D. J. Malan, "Containerizing CS50: Standardizing Students' Programming Environments," in *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1*, ser. ITiCSE 2024. Association for Computing Machinery, pp. 534–540. [Online]. Available: https://dl.acm.org/doi/10.1145/3649217.3653567

[4] E. Dolstra, M. De Jonge, and E. Visser, "Nix: A Safe and Policy-Free System for Software Deployment," pp. 79–92.

[5] E. Dolstra, *The Purely Functional Software Deployment Model*. Utrecht University.

[6] R. Yin. OS as Code - My Experience of NixOS. This Cute World. [Online]. Available: https://thiscute.world/en/posts/my-experience-of-nixos/

[7] D. Holland, W. Josephson, K. Magoutis, M. Seltzer, C. Stein, and A. Lim, "Research issues in No-Futz computing," in *Proceedings Eighth Workshop on Hot Topics in Operating Systems*, pp. 106–110. [Online]. Available: https://ieeexplore.ieee.org/document/990069/

[8] W. Du, J.-M. Gratien, R. Gayno, and B. Raffin. Nix to the Rescue for a Reproducible HPC-AI Software Stack. [Online]. Available: http://arxiv.org/abs/2607.26688

[9] X. Iaso. Nix is a better Docker image builder than Docker's image builder. [Online]. Available: https://xeiaso.net/talks/2024/nix-docker-build/

[10] E. Dolstra and A. Hemel, "Purely Functional System Configuration Management."

[11] E. Dolstra, A. Löh, and N. Pierron, "NixOS: A purely functional Linux distribution," vol. 20, no. 5–6, pp. 577–615. [Online]. Available: https://www.cambridge.org/core/product/identifier/S0956796810000195/type/journal_article

[12] Bandt, "Assessment of Reproducibility in Nix." [Online]. Available: https://zenodo.org/records/17372811

[13] J. Malka, S. Zacchiroli, and T. Zimmermann, "Reproducibility of Build Environments through Space and Time," in *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering:*

New Ideas and Emerging Results. ACM, pp. 97–101. [Online]. Available: https://dl.acm.org/doi/10.1145/3639476.3639767

[14] H. Hasanović. Building Reproducible, Interactive C++ Learning Environments with Nix Flakes and Pixi. [Online]. Available: https://www.linkedin.com/pulse/building-reproducible-interactive-c-learning-nix-flakes-hasanovi%C4%87-tol9e/

[15] Modularity and Lazy Evaluation in NixOS: A Functional Approach to Operating System Configuration. [Online]. Available: https://ieeexplore.ieee.org/document/10991248

[16] M. Hausch, S. Hauser, and B. Uekermann, "Improving reproducibility of scientific software using Nix/NixOS: A case study on the preCICE ecosystem," vol. 83. [Online]. Available: http://eceasst.org/index.php/eceasst/article/view/2613

[17] Q. Meng, Y. Wang, L. Chen, Y. Li, W. Wu, W. Jiang, Q. Wang, C. Lu, Y. Gao, Y. Wu, and Y. Hu. Agent Harness for Large Language Model Agents: A Survey. [Online]. Available: https://www.preprints.org/manuscript/202604.0428

[18] M. A. Forment, M. J. C. Guerrero, F. J. García-Peñalvo, and J. Pereira. The Scaffolding Matters More Than the Interface: A Controlled Comparison of MCP and CLI Tool Use Across Seven Agent Scaffoldings, Five Language Models, and One Software Task. [Online]. Available: http://arxiv.org/abs/2608.08654

[19] P. T. J. Kon, J. Liu, Y. Qiu, W. Fan, T. He, L. Lin, H. Zhang, O. M. Park, G. S. Elengikal, Y. Kang, A. Chen, M. Chowdhury, M. Lee, and X. Wang, "IaC-Eval: A Code Generation Benchmark for Cloud Infrastructure-as-Code Programs."

[20] T. Zhang, S. Pan, Z. Zhang, Z. Xing, and X. Sun. Deployability-Centric Infrastructure-as-Code Generation: Fail, Learn, Refine, and Succeed through LLM-Empowered DevOps Simulation. [Online]. Available: https://arxiv.org/abs/2506.05623

[21] P. Krishna. Importing Phantoms: Measuring LLM Package Hallucination Vulnerabilities. [Online]. Available: https://arxiv.org/abs/2501.19012

[22] K. G. Srivatsa, S. Mukhopadhyay, G. Katrapati, and M. Shrivastava, "A Survey of using Large Language Models for Generating Infrastructure as Code."

[23] Anthropic. Agent Skills. Claude Platform Docs. [Online]. Available: https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview

[24] G. Ling, S. Zhong, and R. Huang. Agent Skills: A Data-Driven Analysis of Claude Skills for Extending Large Language Model Functionality. [Online]. Available: https://arxiv.org/abs/2602.08004

[25] X. Hou, Y. Zhao, S. Wang, and H. Wang. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. [Online]. Available: http://arxiv.org/abs/2503.23278

[26] G. Stoker, T. Arnold, and P. Maxwell, "Using virtual machines to improve learning and save resources in an introductory IT course." ACM, pp. 91–96.

[27] P. Gupta, A. Gabani, C. Nelson, and Y. Shoshitaishvili. CTF Archive: Capture, Curate, Learn Forever. [Online]. Available: http://arxiv.org/abs/2512.01233

[28] J. Malka, S. Zacchiroli, and T. Zimmermann, "Does Functional Package Management Enable Reproducible Builds at Scale? Yes." in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, pp. 775–787. [Online]. Available: https://ieeexplore.ieee.org/document/11025777/

[29] A. Kate, "Generative AI for Infrastructure as Code (IaC): Automating DevOps Configuration, Scripting, and Workflow Management with LLMs."

[30] L. Beurer-Kellner, A. Kudrinskii, M. Milanta, K. B. Nielsen, H. Sarkar, and L. Tal. Technical Report: Exploring the Emerging Threats of the Agent Skill Ecosystem. [Online]. Available: http://arxiv.org/abs/2605.28588

[31] D. Bork and P. Langer, "Language Server Protocol: An Introduction to the Protocol, its Use, and Adoption for Web Modeling Tools," vol. 18, pp. 9:1–16. [Online]. Available: https://emisa-journal.org/emisa/article/view/320

[32] Q. Wu, S. Das, M. Amani, A. Nag, S. Lee, K. P. Gummadi, A. Ravichander, and M. B. Zafar. To Call or Not to Call: A Framework to Assess and Optimize LLM Tool Calling. [Online]. Available: http://arxiv.org/abs/2605.00737

[33] Nixpkgs Manual. [Online]. Available: https://nixos.org/manual/nixpkgs/unstable/#chap-language-support

[34] NixOS Foundation. Content-Addressing File System Objects - Nix 2.35.2 Reference Manual. [Online]. Available: https://nix.dev/manual/nix/2.35/store/file-system-object/content-address#serial-nix-archive

[35] GitHub Actions Hit Three-Nines Failure; One August Outage Consumed Year's Downtime Budget. Tech Times. [Online]. Available: https://www.techtimes.com/articles/324820/20260818/github-actions-hit-three-nines-failure-one-august-outage-consumed-years-downtime-budget.htm

[36] NixOS Wiki contributors. Binary Cache - Official NixOS Wiki. [Online]. Available: https://wiki.nixos.org/wiki/Binary_Cache#Using_a_binary_cache

[37] Garnix. Garnix Blog: Stop trusting Nix caches. [Online]. Available: https://garnix.io/blog/stop-trusting-nix-caches/

[38] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating Large Language Models Trained on Code. [Online]. Available: http://arxiv.org/abs/2107.03374

[39] S. Gooding. Devenv Faces Backlash Over AI-Driven Telemetry in Version 1.4, Prompting Feature Removal. [Online]. Available: https://socket.dev/blog/devenv-

faces-backlash-over-ai-driven-telemetry-in-version-1-4-1

[40] Dwarves Foundation. Nix is faster than Docker build. Dwarves Memo. [Online]. Available: https://memo.d.foundation/topics/devbox/research/nix-is-faster-than-docker-build

[41] E. T. Straub, "Understanding Technology Adoption: Theory and Future Directions for Informal Learning," vol. 79, no. 2, pp. 625–649. [Online]. Available: https://doi.org/10.3102/0034654308325896

[42] Homebrew contributors. FAQ (Frequently Asked Questions). Homebrew Documentation. [Online]. Available: https://docs.brew.sh/FAQ

[43] NixOS Contributors. Tracking: Packages to be orphaned (please consider adopting) · Issue #576945 · NixOS/nixpkgs. GitHub. [Online]. Available: https://github.com/NixOS/nixpkgs/issues/576945

[44] P. Przymus and T. Durieux. Wolves in the Repository: A Software Engineering Analysis of the XZ Utils Supply Chain Attack. [Online]. Available: http://arxiv.org/abs/2504.17473

[45] G. Christensen. Introducing Nixpkgs cooldowns. Determinate Systems. [Online]. Available: https://determinate.systems/blog/nixpkgs-cooldown/